DSA03 – NATURAL LANGUAGE PROCESSING

ASSIGNMENT 1 – CO1 : SOLUTION

Morphology-Aware Text Normalisation and Search Layer for a Hospital Record System

Regular Expressions, Finite-State Automata, Morphological Parsing and the Porter Stemmer

Table 1: Submission details

Field	Details
Student name	M.RAJA MANOHAR REDDY
Register number	192325062
Course	DSA03 – Natural Language Processing
Assignment	Assignment 1 – CO1 (Individual, 1 week, 50 marks)
Submission date	4-11-26
Corpus	520 synthetic, de-identified records; 4 record types; 25,943 words (34,023 non-punctuation tokens)
Code	Python 3.12, standard library plus NLTK (used only to cross-check my own Porter implementation)

Table 2: Mark sheet (to be filled by the evaluator)

Task	Max	Awarded
1 Frame the problem	3	
2 Basic regular expression patterns	4	
3 Tokenisation and normalisation	4	
4 Finite-state automata	5	
5 Inflectional morphology	3	
6 Derivational morphology	4	
7 Finite-state morphological parsing	5	
8 Orthographic rules and two-level morphology	4	
9 The Porter stemmer	5	
10 Implement and test the pipeline	3	
11 Compare, recommend and defend	2	

Task	Max	Awarded
12 Day-4 design challenge	8	
Total	**50**	

Evidence check

Table 3: Where each piece of expected evidence is found

Task	Evidence expected	Where it is in this document
1	Reasoned problem framing: levels, model families, staged pipeline	Task 1 answer, tables and figures under that task
2	Pattern inventory with match specifications and per-pattern error analysis	Task 2 answer, tables and figures under that task
3	Working regex tokenizer/normaliser, examples, limits of regularity	Task 3 answer, tables and figures under that task
4	Automata with subset-construction and minimisation tables; two-way equivalence; non-regularity proof	Task 4 answer, tables and figures under that task
5	Inflectional rules, paradigms, irregular list, measured effect	Task 5 answer, tables and figures under that task
6	Affix inventory, productivity, conflation position	Task 6 answer, tables and figures under that task
7	Transducer specification with traced analyses in both directions and ambiguity	Task 7 answer, tables and figures under that task
8	Rule transducers, composition order, tested cascade with error accounting	Task 8 answer, tables and figures under that task
9	Porter hand-traces, error analysis, retrieval experiment	Task 9 answer, tables and figures under that task
10	Working code, test procedure, result tables and plots	Task 10 answer, tables and figures under that task
11	Justified recommendation from measurements	Task 11 answer, tables and figures under that task
12	Impact analysis, evaluation, redesign, defence	Task 12 answer, tables and figures under that task

Table 4: Implementation evidence chain

Stage	Module (delivered with this report)	Status
Corpus generator (gold entities and gold analyses)	corpus.py	Done
12 entity patterns and evaluation	patterns.py, evalpat.py	Done
Tokenizer and normaliser	tokenizer.py	Done
DFA/NFA, subset construction, minimisation, Thompson, state elimination	automata.py	Done
Lexicon (417 stems), affixes, irregular forms	lexicon.py	Done
Lexicon FST, 7 rule FSTs, composition, parse and generate	morph.py	Done
Porter stemmer with trace	porter.py	Done
Experiments (Tasks 2–11)	exp1.py, extra.py	Done
Day-4 experiments (Task 12)	exp12.py	Done

Table 5: Submission checklist

Item	Yes / No
All 12 tasks answered with evidence	Yes
Corpus size and composition documented (Task 1 and Section 'Data')	Yes
Manual effort accounted for (Tasks 11 and 12)	Yes
Figures and tables are numbered and referred to in the text	Yes
Code runs with the standard library plus NLTK only	Yes
Limitations stated honestly (synthetic corpus, small query set)	Yes

Data and method note (read first)

The assignment allows real or synthetic records provided they are documented. I generated the corpus with a seeded script (corpus.py, seed 7). It contains **520 de-identified records** (130 each of discharge summaries, prescriptions, operation notes and nursing observations), **25,943 words** and 34,023 non-punctuation tokens, and carries **5,310 gold-labelled entities** of the twelve required types. Each record also stores the gold lexical analysis of every content word the generator produced. Because the generator draws from the same lexicon family that my analyser uses, analyser coverage on this corpus is higher than it would be on real hospital text. I say so wherever it matters and I add deliberate out-of-lexicon words (about 22% of symptom slots) and deliberately hard distractors (vitamin B12, laboratory accession numbers, the BD Venflon cannula) so that every pattern has real error cases.

All numbers in this report were produced by the code, not typed by hand. Body text is set in 14 pt and sub-headings in 16 pt; table and code text is smaller so that wide tables fit on the page.

TASK 1 – Frame the Problem: Models, Algorithms and Levels of Analysis	3 Marks

Ans:

Framing the hospital search problem

1. Levels of linguistic analysis

Record search touches only the lower levels. Table 1 maps each level to an example from my corpus and states whether the pipeline needs it.

Table 6: Levels of analysis and the record-search problem

Level	Corpus example	Needed?
Phonology	Clinicians never speak the records; only spelling matters. The one phonological trace is spelling variation such as *anaemia / anemia*.	No (ignored)
Morphology	*fractures, fractured, fracturing* must match a query for *fracture*; *nephrectomies* must relate to *nephrectomy*.	**Yes – core**
Syntax	*The wound was dressed* – the form *dressed* is a participle; *saw* is a noun in one note and a verb in another. Only part-of-speech level information is needed, not full parses.	Marginal
Semantics	*renal* and *kidney* mean the same organ; *hypertension* and *hypotension* are opposites.	Partly (word-sense links in the lexicon)
Pragmatics	*No fever* still contains the word *fever*; a search for fever would return it. Negation and hedging are outside the scope.	No (known limit)
Discourse	*He* in *He was discharged* refers to the patient of the record. Records are short and each concerns one patient.	No

2. Three model families

Table 7: State machines, formal rule systems and probabilistic models in this project

Family	Best-suited task here	Why the other two are the wrong choice
State machines (DFA, NFA, FST)	Validating a patient identifier or an ICD code; mapping *fractures* to *fracture +N +PL* and back	Rewrite rules alone cannot be run backwards; a probabilistic model would give a 97% answer where a patient identifier is either valid or not.
Formal rule systems (rewrite rules, grammars)	The Porter stemmer: ordered suffix-rewrite rules with conditions on the measure m	A transducer would need a lexicon the stemmer is meant to avoid; a probabilistic model would need annotated data that does not exist for the task.

Family	Best-suited task here	Why the other two are the wrong choice
Probabilistic models	Choosing between the analyses of an ambiguous form such as *leaves*, and identifying the language of a token in Task 12	A state machine has no way to rank two legal analyses; a rule system hard-codes a preference that depends on context.

3. Ambiguity

Ambiguity is the central problem because every earlier decision (is this a date, is this a word, which stem?) is made on strings that admit more than one reading, and a wrong early choice cannot be repaired later. I found all four requested kinds in the corpus.

Table 8: Ambiguities found in the corpus

Kind	Example	Readings
Lexical	*discharge*	The heading *Discharge summary* (release from hospital) versus wound *discharge* (fluid). *lead* is an element or a verb.
Morphological	*leaves, saw, left, drains, dressing*	*leaves* = leaf +N +PL or leave +V +3SG; *saw* = noun or past tense of *see*. 72 word types in the corpus have more than one analysis.
Segmentation	*1-0-1, Bed 12, x 5 days, post-operative*	*1-0-1* is one frequency token, not a range or three numbers; *post-operative* is one word or two.
Abbreviation	*MS, BD, OD, DM*	*MS* = multiple sclerosis, mitral stenosis or morphine sulphate (47 occurrences); *BD* = twice daily or the BD Venflon cannula maker (28 false matches in Task 2).

4. Where search appears and what it costs

Table 9: Search in this unit and the resources each use

Search	Where it appears	Time	Memory	Annotated data	Expert effort
DFA run	Validators of Task 4	O(n)	O(1)	None	Design each machine
NFA state-set simulation	Phone NFA, Thompson NFA	O(n·|Q|)	O(|Q|)	None	Design
Subset construction	NFA to DFA	Up to 2^|Q| states; 16 states here	Transition table	None	Low
Backtracking regex matching	Python re on the 12 patterns	Worst case exponential; measured 0.08 s for 520 records	Stack	None	Pattern writing, testing
FST analysis (up)	Composed lexicon-rule machine	Depth-first over 6736 states; 0.020 s for 382 types	8863 arcs	200 annotated forms to test	Lexicon of 417 stems, 7 rules

Search	Where it appears	Time	Memory	Annotated data	Expert effort
Edit-distance candidate search	Task 12	O(m·n) per pair	Small	Confusion weights	Low

5. The pipeline

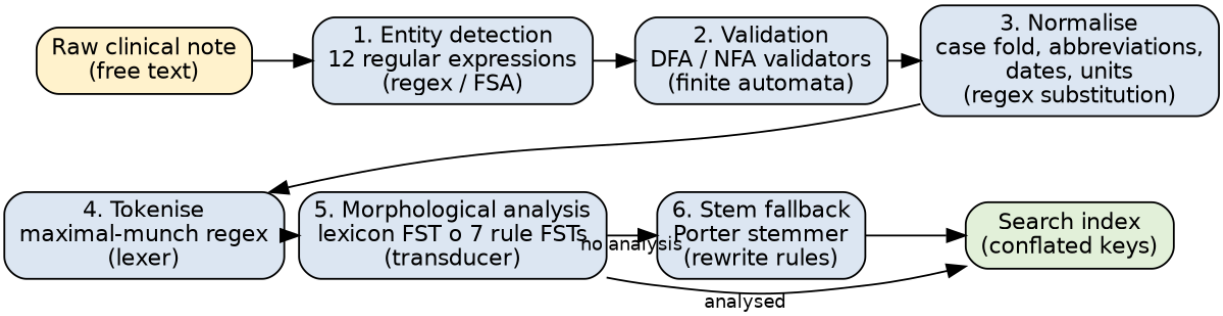


Figure 1: The staged pipeline: input, output and formalism of every stage

Each stage consumes the output of the previous one: raw text enters entity detection (regular expressions); detected entities are validated by automata and masked; the remaining text is normalised and tokenized; each word goes to the finite-state analyser; words with no analysis fall back to the Porter stemmer; the resulting keys form the search index.

TASK 2 – Basic Regular Expression Patterns | 4 Marks

Ans:

Regular expression patterns for twelve clinical entity types

1. Basic operators with clinical examples

Table 10: Operators and a minimal clinical example of each

Operator	Meaning	Example	Matches
Concatenation	one after another	Hb\s\d	Hb 9
Disjunction	either alternative	OD|BD	OD, BD
Kleene star	zero or more	\d*	'', 5, 500
Kleene plus	one or more	\d+\s?mg	5 mg, 500mg
Optionality	zero or one	tablets?	tablet, tablets
Character class	any one of	[CF]	C, F
Range	span in a class	[0-9]	any digit
Negated class	anything except	[^0-9]	K, ., space
Anchors	position, not character	^PID, \bBP\b	PID at line start; BP as a whole word
Counter	exact repetition	\d{2,3}	72, 120
Grouping	scope for operators	(?:mg|g)/dL	mg/dL, g/dL

2. The twelve patterns

Table 11: Pattern inventory: regular expression and match specification

Entity	Regular expression	Must match	Must not match
Patient identifier	\bPID[-/]\d{4}[-/]\d{6}\b	PID-2024-045321, PID/2023/000012	PID-2024-04532 (5 digits); LAB-2024-045321
Admission / discharge date	\b(?:Admitted|Admission|Discharged|Discharge|DOA|DOD)(?:\s+(?:on|date|date\s+of))?\s*[:\-]?\s*(?:(?:0?[1-9]|[12]\d|3[01])[/-](?:0?[1-9]|1[0-2])[/-](?:19|20)\d{2}|(?:0?[1-9]|[12]\d|3[01])-(?:Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec)-(?:19|20)\d{2}|(?:19|20)\d{2}-\d{2}-\d{2})(?![\d/-])	Admitted on 12/03/2024; DOD: 2025-02-18	a bare date with no admission/discharge keyword; 32/01/2024
Drug with strength	\b(?:(?:Tab|Cap|Inj|Syp|Oint)\.?\s+)?[A-Z][a-z]{3,}(?:\s[A-Z][a-z]+)?\s+\d+(?:\.\d+)?\s?(?:mg|mcg|g|IU|ml)(?:/\d+(?:\.\d+)?\s?ml)?\b	Tab. Paracetamol 500 mg; Inj. Ceftriaxone 1 g	Hb 11.2 g/dL (a lab value)
Dosage frequency	\b(?:\d-\d-\d(?:-\d)?|OD|BD|BID|TDS|TID|QID|QDS|SOS|HS|PRN|STAT|(?:once|twice|thrice)\s+(?:a\s+day|daily))\b	1-0-1, BD, SOS, twice daily	ODD, BDS (inside words)
Laboratory value with unit	\b(?:Hb|HbA1c|WBC|Platelets|Creatinine|Urea|Na|K|Glucose|Bilirubin|CRP|ESR|INR)\s*[:=]?\s*\d+(?:\.\d+)?\s*(?:g/dL|mg/dL|mmol/L|%|/cumm|mm/hr|mg/L|U/L)(?![\w/])	Hb 11.2 g/dL; Na 138 mmol/L	Hb 11.2 (no unit)

Entity	Regular expression	Must match	Must not match
Blood pressure	(?:\bBP\s*[:=]?\s*\d{2,3}\s*/\s*\d{2,3}(?:\s*mm\s?Hg)?|(?<![\d/])\d{2,3}\s*/\s*\d{2,3}\s*mm\s?Hg)\b	BP 120/80 mmHg; 130/90 mm Hg	12/03/2024 (a date); ratio 120/80 with no keyword or unit
Temperature	(?:\bTemp(?:erature)?\.?\s*[:=]?\s*)?(?<![\d.])(?:(?:9[5-9]|10[0-5])(?:\.\d)?\s?°\s?F|(?:3[5-9]|4[0-2])(?:\.\d)?\s?°\s?C)(?![A-Za-z])	Temp 98.6 °F; 37.2 °C	Na 138 mmol/L
ICD code	\b[A-TV-Z]\d{2}(?:\.\d{1,4})?[A-Z]?\b	N18.3, I10, S72.001A	B12, T4 (vitamins)
Ward and bed	\b(?:Ward\s*\d{1,2}[A-Z]?|ICU|HDU)[,\s]+Bed\s*\d{1,3}\b|\bW-?\d{1,2}[A-Z]?/B-?\d{1,3}\b	Ward 4B, Bed 12; W-4B/B-12	Bed 12 alone
Contact number	(?<![\d])(?:\+91[\s-]?)?[6-9]\d{4}[\s-]?\d{5}(?![\d])	+91 98765 43210; 9876543210	a 9-digit number; accession number 6234567890
Age	\b\d{1,3}[-]?(?:year|yr)s?[-]?old\b|\b(?:[Aa]ged?:?)\s*\d{1,3}\b|\b\d{1,3}\s?(?:yrs?|y)\s*/\s*[MF]\b	45-year-old; aged 45; 45 yrs/M	for 5 years (a duration)
Duration of symptom	\b(?:for|since|x)\s+(?:the\s+(?:last|past)\s+)?\d+\s*(?:hours?|hrs?|days?|weeks?|months?|years?)\b	for 3 days; since 2 weeks; x 5 days	after 2 weeks (no for/since/x)

Date patterns use the keyword as an anchor, so only admission and discharge dates are labelled and a surgery date in the same record is deliberately not matched. Drug names are matched as a capitalised word of at least four letters followed by a number and a unit.

3. Greedy and non-greedy matching

Sentence: *Tab. Paracetamol 500 mg 1-0-1 and Tab. Amoxicillin 250 mg 1-1-1 for 5 days*. The pattern `Tab\..*mg` (greedy) returns **Tab. Paracetamol 500 mg 1-0-1 and Tab. Amoxicillin 250 mg**. The pattern `Tab\..*?mg` (non-greedy) returns **Tab. Paracetamol 500 mg**.

The star is greedy: it first consumes everything to the end of the string, then backs up one character at a time until the final `mg` can match, so the span covers both drugs and the text between them. The lazy quantifier `*?` does the opposite: it starts with zero characters and extends one character at a time, stopping at the first `mg`, so the span covers only the first drug. For extracting one dosage expression the non-greedy form is the correct one.

4. Precedence and grouping

Precedence from highest to lowest is: parentheses, counters and star/plus/optional, concatenation, disjunction. Disjunction binds loosest, so `\bOD|BD\b` means `\bOD` **or** `BD\b`, not a whole-word `OD` or `BD`. On the string *BDS BODY OD BD ODD ABD* it returned ['OD', 'BD', 'OD', 'BD'] (the extra `OD` is the start of *ODD* and the extra `BD` is the end of *ABD*), while the intended

\b(?:OD|BD)\b returned ['OD', 'BD']. A single misplaced group therefore silently enlarges the language.

5. Evaluation of every pattern over the corpus

Table 12: True positives, false positives, false negatives and causes (exact-span matching)

Pattern	TP	FP	FN	P	R	False-negative cause	False-positive cause
PID	499	0	21	1.000	0.960	separator variant (spaces instead of - or /) (21)	none
DATE	228	0	25	1.000	0.901	ordinal / month-name date (12th March 2024) (25)	none
DRUG	751	73	93	0.911	0.890	lower-case form word or unit glued to number (71); abbreviated drug name (MS) (22)	lab value read as drug (Glucose 281 mg/dL) (73)
FREQ	818	28	26	0.967	0.969	spelled-out frequency (three times a day, every 8 hours) (26)	BD = manufacturer (BD Venflon) (28)
LAB	330	0	1	1.000	0.997	non-listed unit (gm%) (1)	none
BP	365	0	25	1.000	0.936	spoken form (120 by 80) (25)	none
TEMP	356	0	34	1.000	0.913	unit written as 'deg C' (34)	none
ICD	130	55	0	0.703	1.000	none	vitamin name Bnn (B12) (55)
WARDBED	365	0	25	1.000	0.936	reversed order (bed no. 12 in ward 4b) (25)	none
PHONE	152	84	30	0.644	0.835	landline with STD code (30)	10-digit laboratory accession number (84)
AGE	260	0	0	1.000	1.000	none	none
DURATION	875	0	144	1.000	0.859	number written as word (72); slash notation 3/7 (72)	none

Overall micro-precision is **0.955** and micro-recall **0.924** (TP 5129, FP 240, FN 424).

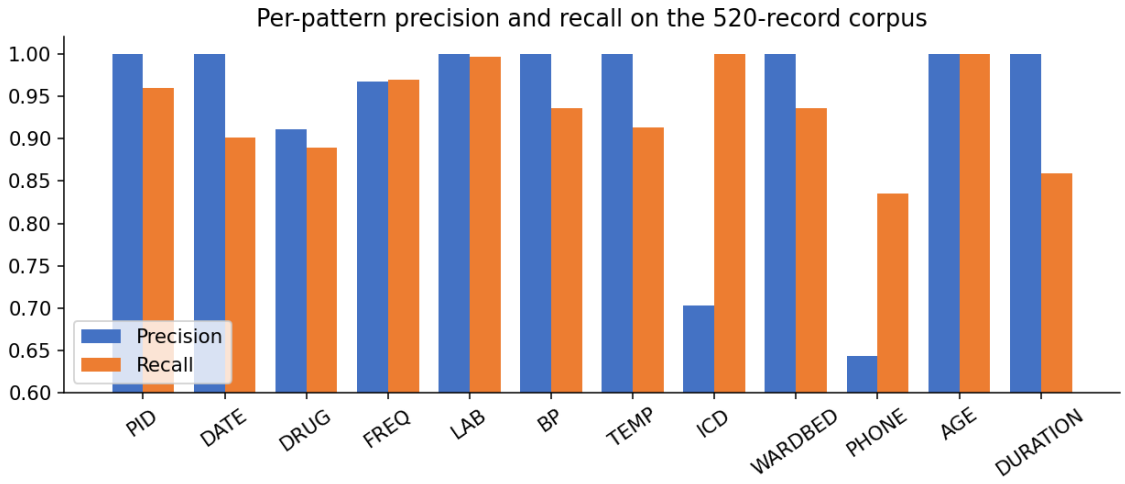


Figure 2: Per-pattern precision and recall

Every error has a linguistic cause rather than a random one. False negatives come from **format variants** that the pattern author did not list (ordinal dates, spelled-out numbers, landline numbers, *deg C*). False positives come from **homography across entity types**: a lab value with a *mg* unit looks like a drug, *B12* looks like an ICD code, a ten-digit laboratory accession number looks like a telephone number, and *BD* is both a frequency and a manufacturer. These are the limits of purely orthographic matching.

TASK 3 – Regular Expressions for Tokenisation and Normalisation	4 Marks

Ans:

A regular-expression tokenizer and normaliser

1. The tokenizer as one regular expression

Table 13: Token classes (alternatives of the single expression)

Class	Regular expression	Decision and justification
DATE	\d{1,2}[/-]\d{1,2}[/-]\d{4}|\d{1,2}-[A-Za-z]{3}-\d{4}|\d{4}-\d{2}-\d{2}	One token so the date can be canonicalised as a unit
FREQ	\d-\d-\d(?:-\d)?	1-0-1 is a single dosage code, not a range
RATIO	\d{2,3}\s?/\s?\d{2,3}	Blood-pressure style pairs stay whole
DECIMAL	\d+\.\d+	11.2 must not be split at the full stop
NUMBER	\d+(?:,\d{3})*	Thousands separators kept inside the number
ABBRSL	(?:[A-Za-z]/){1,2}[A-Za-z]{1,3}\b	c/o, h/o, k/c/o, N/V are single abbreviations
ABBRDOT	(?:[A-Za-z]\.){2,}|(?:Dr|Tab|Cap|Inj|Syp|Oint|No)\.	b.i.d. and Tab. keep their full stops
HYPHEN	[A-Za-z]+(?:-[A-Za-z]+)+	post-operative, non-healing are one term
WORD	[A-Za-z]+(?:'[a-z]+)?	Ordinary words, possessives kept
PUNCT	[^\sA-Za-z0-9]	Anything else is a one-character token

Units written against numbers (*500mg*) are deliberately split by this expression into the number 500 and the word mg, because the unit then becomes a separate token that the normaliser can canonicalise.

2. Maximal munch compared with whitespace splitting

My tokenizer tries every alternative at the current position and keeps the **longest** match (a lexer, not Python's leftmost-first alternation).

Table 14: Maximal-munch tokens against a whitespace split

Input	Maximal munch	Whitespace split
Tab. Paracetamol 500mg 1-0-1	Tab. | Paracetamol | 500 | mg | 1-0-1	Tab. | Paracetamol | 500mg | 1-0-1
Hb 11.2 g/dL, c/o N/V	Hb | 11.2 | g/dL | , | c/o | N/V	Hb | 11.2 | g/dL, | c/o | N/V
post-operative b.i.d. k/c/o DM	post-operative | b.i.d. | k/c/o | DM	post-operative | b.i.d. | k/c/o | DM

The whitespace split glues *500mg* into one token, leaves *c/o,* and *g/dL,* with trailing commas, and cannot tell that *k/c/o* is one unit. Maximal munch keeps *1-0-1*, *11.2* and *b.i.d.* whole and separates punctuation.

3. Normalisation rules and a full before-and-after record

Normalisation applies, in order: (1) expansion of 43 clinical abbreviations (listed below), (2) canonicalisation of dates to ISO form, (3) unit canonicalisation (*500mg → 500 mg*, *mm Hg → mmHg*, *gm% → g/dL*, *°C → degC*), (4) removal of the hyphen after the prefixes post-, pre-, non-, intra-, peri-, anti-, hyper-, hypo-, re-, un-, sub-, (5) case folding.

Table 15: Abbreviation expansion table (43 abbreviations)

Abbreviation	Expansion	Abbreviation	Expansion
BPH	benign prostatic hyperplasia	MRI	magnetic resonance imaging
CAD	coronary artery disease	N/V	nausea and vomiting
CBC	complete blood count	NPO	nil by mouth
CKD	chronic kidney disease	OPD	outpatient department
COPD	chronic obstructive pulmonary disease	OT	operation theatre
CT	computed tomography	PO	by mouth
CVA	cerebrovascular accident	RFT	renal function test
DM	diabetes mellitus	Rx	prescription
Dx	diagnosis	SOB	shortness of breath
ECG	electrocardiogram	Sx	symptoms
Fx	fracture	TURP	transurethral resection of prostate
GERD	gastro-oesophageal reflux disease	Tx	treatment
HDU	high dependency unit	URTI	upper respiratory tract infection
HTN	hypertension	USG	ultrasonography
Hx	history	UTI	urinary tract infection
ICU	intensive care unit	c/o	complains of
ID	identity	h/o	history of
IM	intramuscular	k/c/o	known case of
IV	intravenous	pt	patient
LFT	liver function test	s/p	status post
LSCS	lower segment caesarean section	w/u	work up
MI	myocardial infarction		

Before:

OPERATION NOTE. PID-2023-687888. Procedure: laparoscopy. Indication: allergy for the past 8 hours. Anaesthesia: general. The joint was sutured and incision was closed with sutures. A drain was placed. Blood loss was minimal. Specimen sent for culture. Vitals: 184/81 mmHg, Temp 99.1 °F. Shifted to bed no. 39 in ward 3c. Postoperative monitoring in recovery. Notes: liver, inflammation, toxicity.

After:

operation note. pid-2023-687888. procedure: laparoscopy. indication: allergy for the past 8 hours. anaesthesia: general. the joint was sutured and incision was closed with sutures. a drain was placed. blood loss was minimal. specimen sent for culture. vitals: 184/81 mmhg, temp 99.1 degf. shifted to bed no. 39 in ward 3c. postoperative monitoring in recovery. notes: liver, inflammation, toxicity.

Over the whole corpus the normaliser expanded 644 abbreviations, canonicalised 188 dates and 153 unit strings.

4. Irreversible normalisation decisions

Table 16: Information destroyed by normalisation

Decision	What is lost	Acceptable when	Not acceptable when
Case folding	*HTN* versus *htn*; *US* (ultrasound) versus *us*	Matching ordinary words	Reading abbreviations whose meaning depends on case
Abbreviation expansion	The original ambiguous form; *DM* and *MS* have several expansions	Unambiguous abbreviations (*c/o*)	Ambiguous ones – I therefore leave *MS*, *PE* and *RA* unexpanded
Date canonicalisation	Original layout, and the dd/mm versus mm/dd convention of the writer	The source convention is known and fixed	Records from mixed conventions, where 03/04/2024 is ambiguous
Hyphen removal after prefixes	Visible prefix boundary	Search	When the hyphen separates two terms (*non-healing*)

5. What no regular expression can express

Matching **balanced, nested brackets** in a clinical note, such as *(Dx: HTN (on Tab. X (BD)))*, is not regular. A finite automaton has a fixed number of states, but recognising that every (is closed requires counting arbitrarily deep nesting. Task 4 proves this with the pumping lemma. In practice a regular layer can match one fixed depth of nesting but not the general case.

TASK 4 – Finite State Automata: Design, Non-determinism and Equivalence	5 Marks

Ans:

Automata for the entity validators

1. Formal definition and four deterministic validators

A finite automaton is a five-tuple **(Q, Σ, δ, q_0, F)**: a finite set of states Q, an input alphabet Σ, a transition function δ: Q × Σ → Q (for an NFA δ: Q × (Σ ∪ {ε}) → 2^Q), a start state $q_0 \in Q$ and a set of accepting states $F \subseteq Q$. A string is accepted if the machine ends in F after reading it; a missing transition goes to an implicit dead state.

To keep tables readable the machines work over **character classes**: *d* = any digit, *L* = a capital letter A–T or V–Z, *s* = hyphen or slash, and for dates Z=0, O=1, T=2, H=3, L=4–9, S=separator. The star (*) marks accepting states.

Validator 1 – patient identifier

Format PID s four digits s six digits (for example PID-2024-045321). 16 states; every state has exactly one outgoing live transition, so the machine is a simple chain and is already minimal.

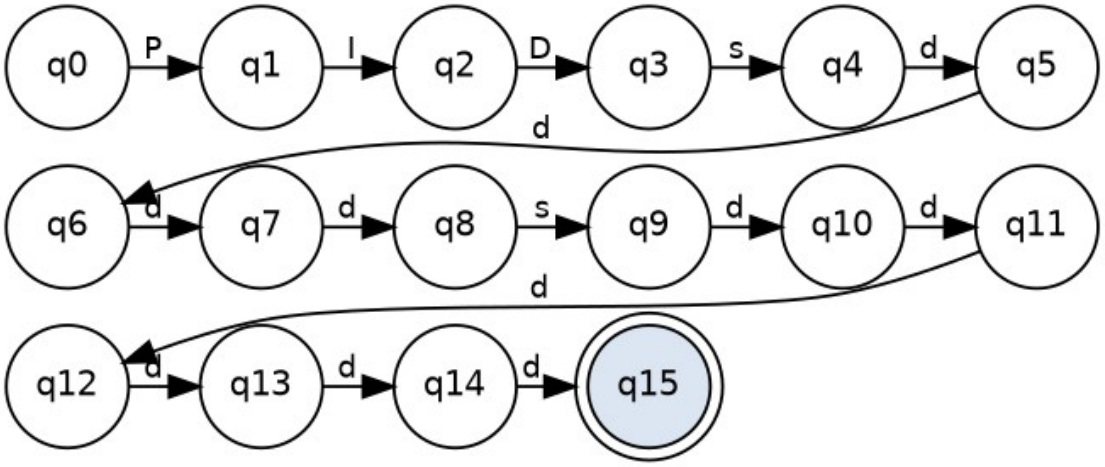


Figure 3: Transition diagram of the PID validator

Table 17: Transition table of the PID DFA (16 states; minimised: 16)

State	P	I	D	s	d
q0→	q1	–	–	–	–
q1	–	q2	–	–	–
q10	–	–	–	–	q11
q11	–	–	–	–	q12

State	P	I	D	s	d
q12	–	–	–	–	q13
q13	–	–	–	–	q14
q14	–	–	–	–	q15
q15*	–	–	–	–	–
q2	–	–	q3	–	–
q3	–	–	–	q4	–
q4	–	–	–	–	q5
q5	–	–	–	–	q6
q6	–	–	–	–	q7
q7	–	–	–	–	q8
q8	–	–	–	q9	–
q9	–	–	–	–	q10

Validator 2 – ICD code

Format letter, two digits, then an optional point with one to three digits. States q3, q5, q6 and q7 all accept, so *I10*, *N18.3*, *K35.80* and *E11.9* are accepted. A trailing letter such as the *A* of *S72.001A* is deliberately outside this core validator and is rejected (the regular expression of Task 2 allows it).

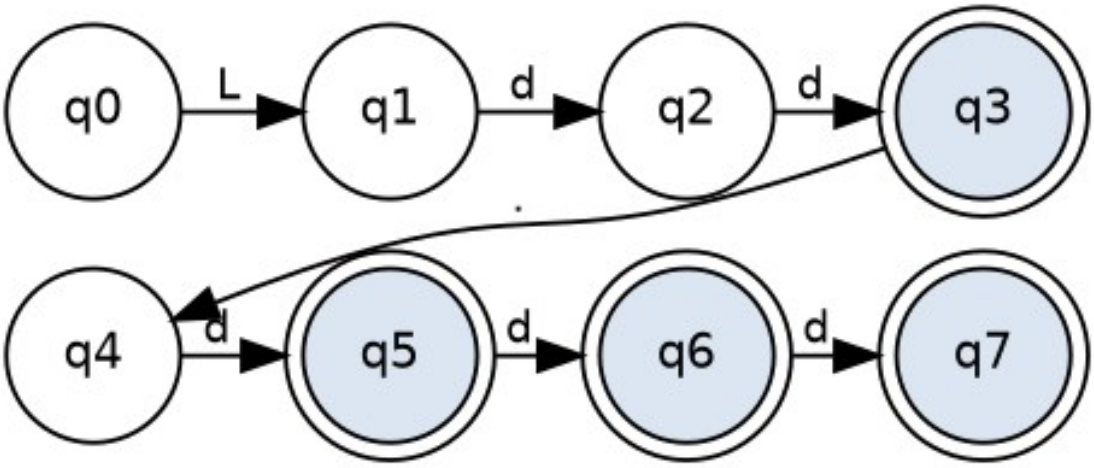


Figure 4: Transition diagram of the ICD validator

Table 18: Transition table of the ICD DFA (8 states; minimised: 8)

State	L	d	.
q0→	q1	–	–
q1	–	q2	–
q2	–	q3	–
q3*	–	–	q4
q4	–	q5	–
q5*	–	q6	–
q6*	–	q7	–

State	L	d	.
q7*	–	–	–

Validator 3 – blood-pressure pair

Format two or three digits, a slash, two or three digits. The branch at q2 (slash after two digits or a third digit) is what the counter {2,3} compiles to.

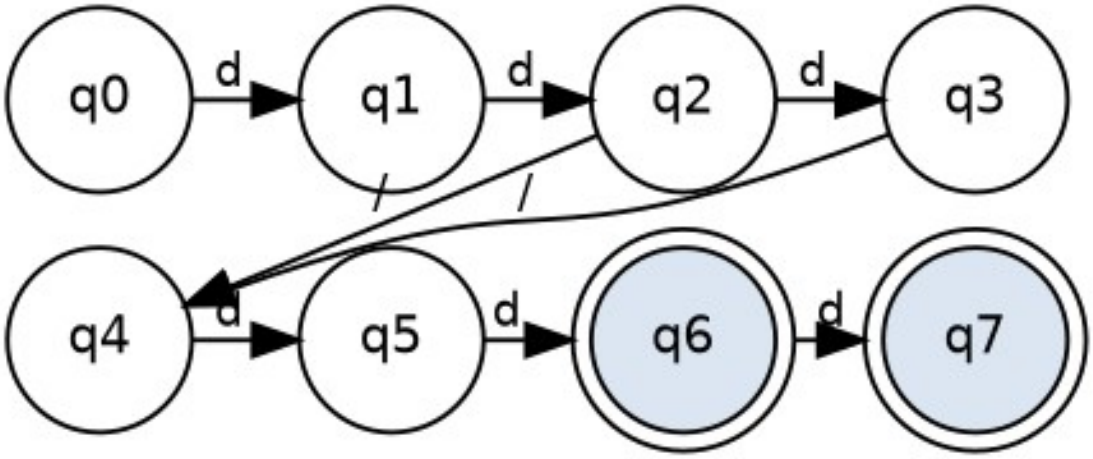


Figure 5: Transition diagram of the BP validator

Table 19: Transition table of the BP DFA (8 states; minimised: 8)

State	d	/
q0→	q1	–
q1	q2	–
q2	q3	q4
q3	–	q4
q4	q5	–
q5	q6	–
q6*	q7	–
q7*	–	–

Validator 4 – calendar date dd/mm/yyyy

This validator checks ranges, not only shape: the day is 01–31 (first digit 0, 1, 2 or 3; after 3 only 0 or 1), the month is 01–12, the year starts with 1 or 2. It still accepts 31/02/2024, because month length is arithmetic, not a regular constraint on digit sequences. Minimisation merges the day states after 1 and after 2 (they behave identically), giving 14 states from 15.

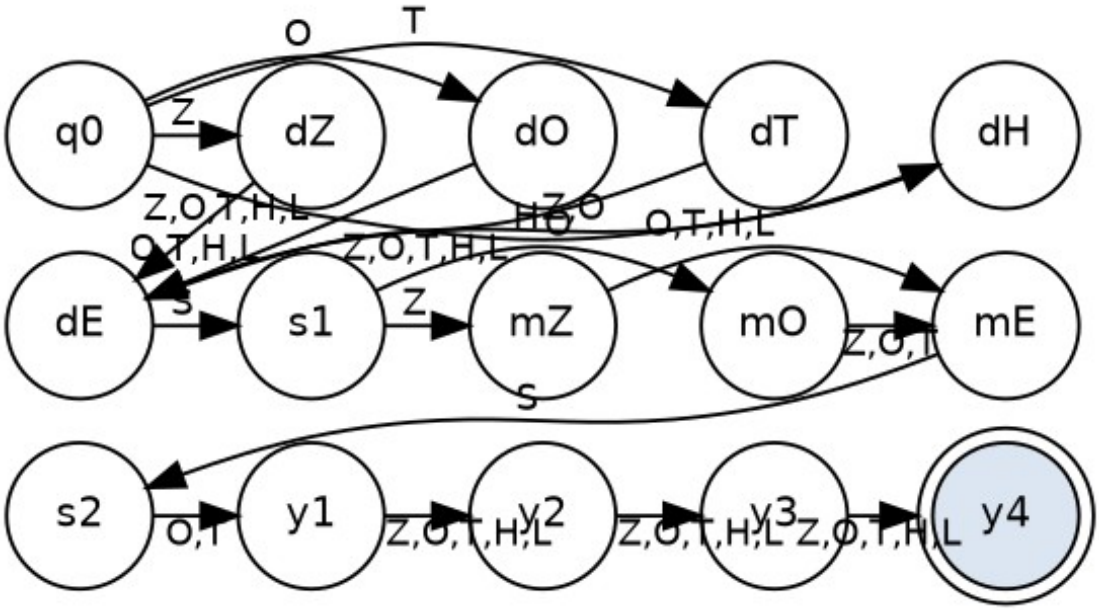


Figure 6: Transition diagram of the DATE validator

Table 20: Transition table of the DATE DFA (15 states; minimised: 14)

State	Z	O	T	H	L	S
dE	–	–	–	–	–	s1
dH	dE	dE	–	–	–	–
dO	dE	dE	dE	dE	dE	–
dT	dE	dE	dE	dE	dE	–
dZ	–	dE	dE	dE	dE	–
mE	–	–	–	–	–	s2
mO	mE	mE	mE	–	–	–
mZ	–	mE	mE	mE	mE	–
q0→	dZ	dO	dT	dH	–	–
s1	mZ	mO	–	–	–	–
s2	–	y1	y1	–	–	–
y1	y2	y2	y2	y2	y2	–
y2	y3	y3	y3	y3	y3	–
y3	y4	y4	y4	y4	y4	–
y4*	–	–	–	–	–	–

2. An NFA with ε-transitions: telephone numbers

A contact number has optional parts: an optional country prefix *+91*, an optional space or hyphen after it, and an optional separator after the fifth digit. Writing these options as ε-transitions is natural. The classes are: +, *9*, *1*, *m* = 6,7,8, *z* = 0,2,3,4,5 and *s* = space or hyphen. A mobile number starts with 6–9, so the first digit class is 9 or m.

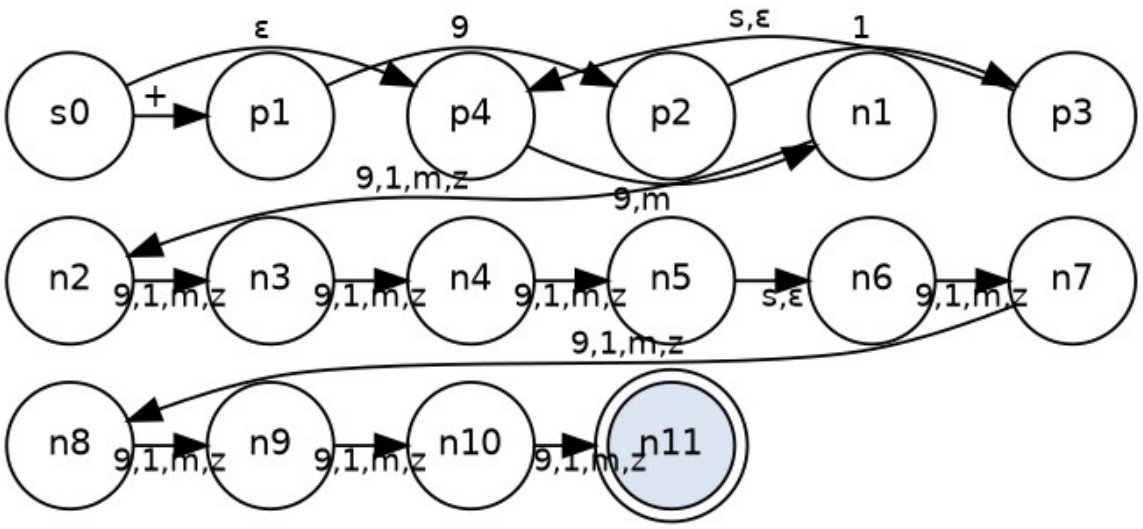


Figure 7: ε-NFA for contact numbers (16 states)

Table 21: Trace of the accepted string +91 98765 43210 (set of live states after each symbol)

Symbol read (class)	Live states
start	{p4, s0}
+→+	{p1}
9→9	{p2}
1→1	{p3, p4}
→s	{p4}
9→9	{n1}
8→m	{n2}
7→m	{n3}
6→m	{n4}
… (digits continue)	…
0→z	{n11} → contains n11, ACCEPT

Table 22: Trace of the rejected string 98765 4321 (only nine digits)

Symbol read (class)	Live states
start	{p4, s0}
9→9	{n1}
8→m	{n2}
7→m	{n3}
6→m	{n4}
5→z	{n5, n6}
→s	{n6}
4→z	{n7}
3→z	{n8}
2→z	{n9}
1→1	{n10}

The second string is rejected because after the last digit the live set is {n10}, which does not contain the accepting state n11.

3. Subset construction, then minimisation

Table 23: Complete subset-construction table (ε-closure applied; Ø = no move)

DFA state	NFA states	+	9	1	m	z	s
D0	p4,s0	D1	D2	Ø	D2	Ø	Ø
D1	p1	Ø	D3	Ø	Ø	Ø	Ø
D2	n1	Ø	D4	D4	D4	D4	Ø
D3	p2	Ø	Ø	D5	Ø	Ø	Ø
D4	n2	Ø	D6	D6	D6	D6	Ø
D5	p3,p4	Ø	D2	Ø	D2	Ø	D7
D6	n3	Ø	D8	D8	D8	D8	Ø
D7	p4	Ø	D2	Ø	D2	Ø	Ø
D8	n4	Ø	D9	D9	D9	D9	Ø
D9	n5,n6	Ø	D10	D10	D10	D10	D11
D10	n7	Ø	D12	D12	D12	D12	Ø
D11	n6	Ø	D10	D10	D10	D10	Ø
D12	n8	Ø	D13	D13	D13	D13	Ø
D13	n9	Ø	D14	D14	D14	D14	Ø
D14	n10	Ø	D15	D15	D15	D15	Ø
D15	n11	Ø	Ø	Ø	Ø	Ø	Ø

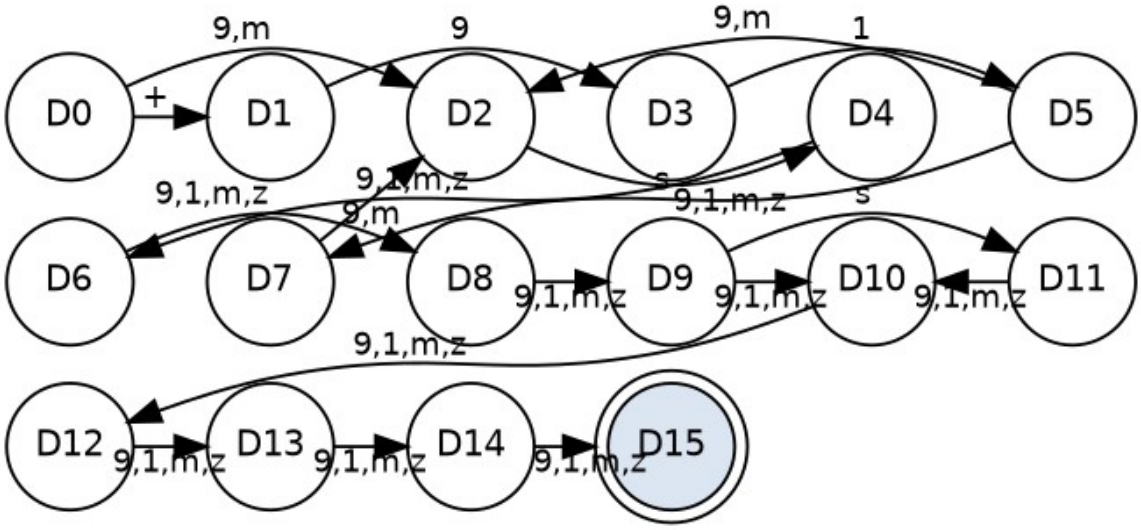


Figure 8: DFA obtained by subset construction (16 states)

The NFA has 16 states and the subset construction also produced **16** reachable DFA states, so removing the ε-transitions cost nothing here: the options are mostly independent and do not interleave. Moore partition refinement on this DFA leaves **16** states (no pair is equivalent because each digit position is a distinct counter value). The worst case for subset construction is 2^|Q|, but

optional parts and counters give a blow-up of 1×. For the date validator minimisation did reduce the machine, from 15 to 14 states.

4. Thompson's construction and the equivalence with regular expressions

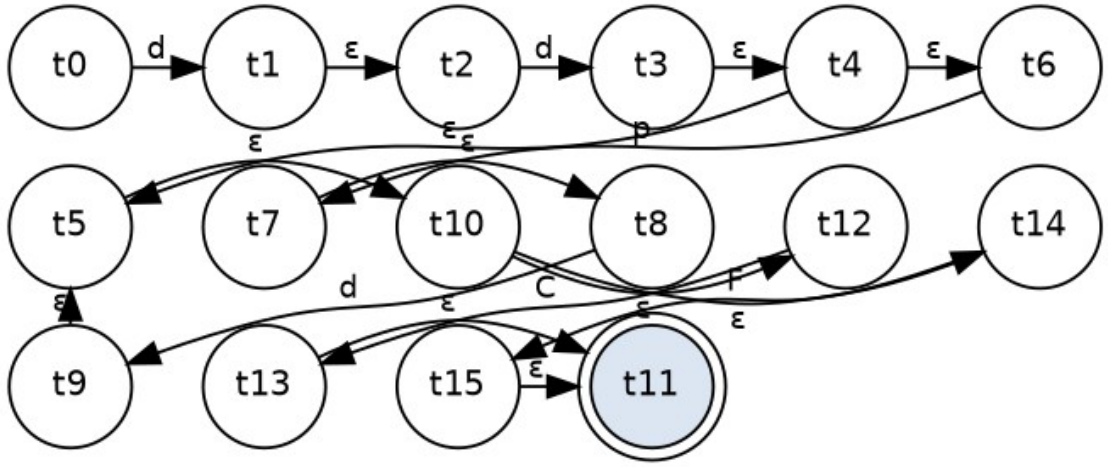


Figure 9: Thompson ε-NFA for the temperature core dd(.d)?[CF] (symbols d, p = point, C, F)

I implemented Thompson's construction (automata.thompson) with one fragment per operator: a symbol gets two states joined by the symbol; concatenation links the end of one fragment to the start of the next by ε; disjunction adds a new start and a new end with ε-branches; star, plus and optional add the ε-loops and bypass. Applied to the temperature regular expression dd(pd)?(C|F) it produced **16** states.

Regular expression → automaton. I compared the Thompson NFA with Python's matcher on every string of length 0–7 over {d, p, C, F}: 21845 strings, **0 disagreements**.

Automaton → regular expression. State elimination (automata.to_regex) on the minimised ICD machine returned (((Ldd|Ldd.d)|Ldd.dd)|Ldd.ddd), which simplifies to L d d (. d (d (d)?)?)?, i.e. the pattern [A-TV-Z]\d{2}(\.\d{1,3})?. Testing the machine against that expression on all 3280 strings of length 0–7 over {L, d, .} gave **0 disagreements**. The equivalence therefore holds in both directions.

5. The pumping lemma and a non-regular clinical pattern

Pumping lemma. If L is regular there is a number $p \geq 1$ such that every string $w \in L$ with $|w| \geq p$ can be written $w = xyz$ with $|xy| \leq p$, $|y| \geq 1$, and $xy^iz \in L$ for every $i \geq 0$.

Claim. The language $L = \{ (^n)^n : n \geq 1 \}$ of correctly closed bracket groups, the simplest form of a nested clinical note such as *(Dx: HTN (on Tab. X))*, is not regular.

Proof. Suppose L were regular with pumping length p. Take $w = (^p)^p \in L$, which has length $2p \geq p$. Any decomposition $w = xyz$ with $|xy| \leq p$ puts y entirely inside the opening brackets, so $y = (^k$ with $k \geq 1$. Pumping with $i = 2$ gives $(^{p+k})^p$, which has more opening than closing brackets and is not in L. This contradicts the lemma, so L is not regular. ■

Link to Task 3. This is exactly why the tokenizer can match a bracketed group of one fixed depth but cannot enforce balanced nesting of arbitrary depth: the automaton would need a counter that grows with n, that is, unbounded memory, which only a pushdown automaton (a context-free grammar) has.

TASK 5 – Inflectional Morphology	3 Marks

Ans:

Inflectional processes in clinical English

1. Definitions and the inflection–derivation criteria

Table 24: Terms

Term	Definition	Example
Morpheme	smallest meaning-bearing unit	*fracture*, *-s*, *-ed*
Stem	the base to which affixes attach	*fracture* in *fractured*
Affix	a bound morpheme (prefix or suffix)	*-ing*, *re-*
Inflection	grammatical variation of one lexeme	*fractures, fractured*
Paradigm	the full set of inflected forms of a lexeme	*bleed, bleeds, bled, bled, bleeding*

Table 25: Inflection versus derivation

Criterion	Inflection	Derivation
Category	Preserves it (*fracture* N → *fractures* N)	Often changes it (*operate* V → *operation* N)
Productivity	Almost fully productive	Partly productive, lexically restricted
Position	Outside derivational affixes (*operation-s*)	Inside inflection (*treat-ment-s*, never *treat-s-ment*)
Meaning	Grammatical function only	New lexeme with new meaning

2. Inflectional paradigms present in the corpus

The corpus contains four kinds: noun number (-s, -es, -ies, irregular), verb tense and aspect (past, past participle, present participle), subject agreement (3rd-person -s) and adjective comparison (-er, -est). the table below lists all paradigms generated **by the transducer itself** (generation direction) for ten lexemes.

Table 26: Full inflectional paradigms produced by the transducer

Lexeme	Feature – form
fracture (N)	SG fracture ; PL fractures
fracture (V)	BASE fracture ; 3SG fractures ; PAST fractured ; PP fractured ; PROG fracturing
dress (V)	BASE dress ; 3SG dresses ; PAST dressed ; PP dressed ; PROG dressing
stop (V)	BASE stop ; 3SG stops ; PAST stopped ; PP stopped ; PROG stopping
bleed (V)	BASE bleed ; 3SG bleeds ; PAST bled ; PP bled ; PROG bleeding
apply (V)	BASE apply ; 3SG applies ; PAST applied ; PP applied ; PROG applying
large (A)	POS large ; COMP larger ; SUP largest
diagnosis (N)	SG diagnosis ; PL diagnoses

Lexeme	Feature – form
child (N)	SG child ; PL children
injury (N)	SG injury ; PL injuries

The affix inventory contains **23** inflectional affixes counting orthographic variants: noun plural -s, -es, -ies, -ves, -a, -ae, -i, -ices, -es(is); possessive 's, s'; verb -s, -es, -ies, -ed, -d, -ied, -ing, -en/-n; adjective -er, -est, -ier/-iest.

3. Orthographic rules and where each over-generates

Table 27: Orthographic rules and counter-examples

Rule (precise statement)	Correct	Over-generation counter-example
E-deletion: e → ε / __ ^ V (delete a stem-final e before a vowel-initial suffix)	fracture+ing → fracturing	see+ing → *seing (should be seeing); dye+ing* → dying; manage+able → *managable
Consonant doubling: C → CC / V C __ ^ {ing, ed, er, est} for stems marked in the lexicon	stop+ed → stopped	Applied to every CVC stem: visit+ing → *visitting; offer+ed* → offerred
Y-to-i: y → i, ie / C __ ^ {ed, er, est, s}	carry+ed → carried; injury+s → injuries	shy+er → *shier (shyer is also correct); Kennedy+s* → Kennedies
Sibilant -es: ε → e / {x, s, z, sh, ch} ^ __ s #	abscess+s → abscesses	stomach+s → *stomaches (ch is pronounced /k/)

4. Irregular forms

The lexicon lists **92 irregular forms of 53 lexemes** separately, because no rule can derive them: they involve vowel change (*take → took*), suppletion (*be → was, were*), Latin and Greek plurals (*bacterium → bacteria, bronchus → bronchi, appendix → appendices, diagnosis → diagnoses*) and zero change (*put, cut, set*). A rule would have to know the individual word, which is exactly what a listing is.

Table 28: Selected irregular forms

Type	Forms
Verbs (past / participle)	was/were/been, had, went/gone, took/taken, gave/given, felt, held, left, found, saw/seen, began/begun, broke/broken, brought, kept, lost, underwent/undergone, wore/worn, slept, ate/eaten, fell/fallen, bled, led, meant, stood, lay/lain, knew/known, sent, came
Zero-change verbs	put, cut, set, spread
Nouns	children, teeth, feet, leaves, knives, men, women
Latin / Greek plurals	bacteria, vertebrae, bronchi, appendices, fungi, femora, diagnoses, analyses, prognoses, dialyses

5. Measured effect of inflection

From the gold analyses of the 6113 content-word tokens that the generator produced, there are **234 word types** for **169 lemmas**, a type-to-lemma ratio of **1.38**. Inflection alone therefore inflates the vocabulary by about 38%.

On the 40 queries, exact matching (no stemming) misses relevant records for **38** queries. Of those, **18** are recovered completely by inflectional lemmatisation alone; over all queries, **438 of 797** missed query–record pairs (55%) are explained by inflection alone. The remainder need derivational or concept links (Task 6).

TASK 6 – Derivational Morphology	4 Marks

Ans:

Derivation in the clinical vocabulary

1. Derivation and the affix inventory

Derivation builds new lexemes from old ones and usually changes the category (*operate* V → *operation* N → *operational* A) or the meaning (*hyper-*, *hypo-*). The inventory below lists the **32 derivational affixes** that I implemented in the transducer and how often each is attested in the corpus (types / tokens).

Table 29: Derivational affix inventory (24 attested in the corpus out of 32 implemented)

Affix	Category change	Meaning	Example	Types / tokens
-ion	V→N	act or result of	operation	8 / 356
-ation	V→N	act or result of	examination	3 / 143
-ive	V→A	tending to	operative	4 / 179
-able	V→A	capable of being	treatable	1 / 6
-ment	V→N	result of	treatment	2 / 14
-ity	A→N	state of	toxicity	1 / 17
-ful	N→A	full of	painful	1 / 9
-less	N→A	without	breathless	1 / 75
-ish	N→A	somewhat	feverish	1 / 8
-ness	A→N	state of	weakness	1 / 10
-ly	A→ADV	manner	severely	1 / 8
-itis	B→N	inflammation of	nephritis	8 / 151
-ectomy	B→N	surgical removal	nephrectomy	12 / 124
-logy	B→N	study of	nephrology	0 (rule only)
-osis	B→N	condition	nephrosis	1 / 20
-oma	B→N	tumour of	hepatoma	2 / 15
-pathy	B→N	disease of	neuropathy	1 / 17
-scopy	B→N	viewing	arthroscopy	4 / 28
-tomy	B→N	incision	craniotomy	4 / 24
-stomy	B→N	opening	colostomy	2 / 11
-plasty	B→N	repair	angioplasty	2 / 16
-algia	B→N	pain in	—	0 (rule only)
-graphy	B→N	imaging	—	0 (rule only)
re-	V→V	again	reoperated	1 / 5
un-	A→A	not	unstable	1 / 12
post-	A→A	after	postoperative	1 / 168
pre-	A→A	before	preoperative	1 / 4
hyper-	N→N	above normal	hypertension	0 (rule only)

Affix	Category change	Meaning	Example	Types / tokens
hypo-	N→N	below normal	hypotension	0 (rule only)
anti-	N→N	against	antibiotic	0 (rule only)
intra-	A→A	inside	intravenous	0 (rule only)
peri-	A→A	around	perioperative	0 (rule only)

24 of the 32 affixes are attested in the corpus, which meets the requirement of at least fifteen. *hyper-* and *hypo-* occur only inside the listed stems *hypertension* and *hypotension*, so the transducer treats them as available prefixes but does not use them to analyse those words.

2. Derivational families and impossible orders

Table 30: Derivational families generated by the transducer (lexical form → surface)

Root	Members
operate	operate +V +BASE → **operate** | operate +V +ION +N +SG → **operation** | operate +V +IVE +A +POS → **operative** | PFX_POST operate +V +IVE +A +POS → **postoperative** | PFX_PRE operate +V +IVE +A +POS → **preoperative** | PFX_RE operate +V +PP → **reoperated**
infect	infect +V +ION +N +SG → **infection** | infect +V +ION +N +PL → **infections** | infect +V +IVE +A +POS → **infective**
treat	treat +V +ABLE +A +POS → **treatable** | PFX_UN treat +V +ABLE +A +POS → **untreatable** | treat +V +MENT +N +SG → **treatment**
nephro-	nephro +B +ITIS +N +SG → **nephritis** | nephro +B +ECTOMY +N +SG → **nephrectomy** | nephro +B +LOGY +N +SG → **nephrology** | nephro +B +OSIS +N +SG → **nephrosis** | nephro +B +PATHY +N +SG → **nephropathy**
hepato-	hepato +B +ITIS +N +SG → **hepatitis** | hepato +B +ECTOMY +N +SG → **hepatectomy** | hepato +B +OMA +N +SG → **hepatoma** | hepato +B +LOGY +N +SG → **hepatology**
examine	examine +V +ATION +N +SG → **examination** | examine +V +ATION +N +PL → **examinations** | examine +V +PP → **examined**
stable	stable +A +POS → **stable** | PFX_UN stable +A +POS → **unstable**

Affix order is fixed: prefix – stem – derivational suffix(es) – inflectional suffix. I asked the transducer to generate five impossible orders and it returned no string for any of them:

Table 31: Impossible orders rejected by the lexicon FST

Attempted lexical string	Why it is impossible	Output
treat +V +PAST +ABLE +A +POS	inflection before derivation	none (rejected)
treat +V +ABLE +A +POS PFX_UN	prefix after the word	none (rejected)
nephro +B +ITIS +N +PL +ITIS	suffix after inflection	none (rejected)
operate +V +ION +N +SG PFX_POST	prefix after suffix	none (rejected)
hepato +B +ITIS +A +POS	-itis cannot yield an adjective	none (rejected)

3. Semantic non-compositionality

A derived word's meaning is not always the sum of its parts. Examples from the clinical vocabulary:

- *dressing* is not *dress* + *-ing*: it is a wound covering. *Dressing* (noun) and *dressed* (verb) must stay separate senses.
- *discharge* is not *dis-* + *charge*: it means release from hospital or fluid from a wound.
- *recovery* is not *re-* + *cover*; *nursing* is not just the act of nursing a patient but a profession and department; *operation* means surgery in a hospital but a *surgical operation* in a general text means something else.
- *medical* and *medication* look related but one concerns the profession and the other a drug.

A search system that stems aggressively strips these affixes and then treats *medical* and *medication* as the same term. The measured consequence appears in Task 9: Porter maps both to *medic*.

4. Productivity of the affixes

Table 32: Types, tokens, hapax legomena and Baayen's productivity P = hapaxes / tokens

Affix	Types	Tokens	Hapax	P	Class
-ion	8	356	0	0.000	productive
-ation	3	143	0	0.000	restricted (few bases)
-ive	4	179	0	0.000	productive
-able	1	6	0	0.000	restricted (few bases)
-ment	2	14	0	0.000	restricted (few bases)
-ity	1	17	0	0.000	restricted (few bases)
-ful	1	9	0	0.000	restricted (few bases)
-less	1	75	0	0.000	restricted (few bases)
-ish	1	8	0	0.000	restricted (few bases)
-ness	1	10	0	0.000	restricted (few bases)
-ly	1	8	0	0.000	restricted (few bases)
-itis	8	151	0	0.000	productive
-ectomy	12	124	2	0.016	productive
-osis	1	20	0	0.000	restricted (few bases)
-oma	2	15	0	0.000	restricted (few bases)
-pathy	1	17	0	0.000	restricted (few bases)
-scopy	4	28	1	0.036	productive
-tomy	4	24	2	0.083	productive

Affix	Types	Tokens	Hapax	P	Class
-stomy	2	11	1	0.091	restricted (few bases)
-plasty	2	16	0	0.000	restricted (few bases)
re-	1	5	0	0.000	restricted (few bases)
un-	1	12	0	0.000	restricted (few bases)
post-	1	168	0	0.000	restricted (few bases)
pre-	1	4	0	0.000	restricted (few bases)

An affix is **productive** if it attaches to many different bases and keeps creating new types (here four or more types), and **fossilised** if it survives only inside listed whole words. The evidence for fossilisation is in the lexicon: *-ic* (diabetic, antibiotic, analgesic, diuretic), *-al* (surgical, clinical) and the irregular *-ion* words (admission, reduction, resolution, absorption) could not be derived by the cascade because the stem changes (*admit* → *admission*, *absorb* → *absorption*), so I list them as stems. The corpus is synthetic, so these counts reflect the generator's choices as well as the language; I treat them as the relative ranking, not as language statistics.

5. Should derivational relatives be conflated for search?

Table 33: Conflation: cases that help and cases that mislead

Case	Effect	Verdict
infection – infective – infect	A clinician searching *infection* also wants *infective* findings	Helps
treat – treatable – treatment	Same topic	Helps
hypertension – hypotension	Prefix stripping would equate opposites	**Misleading** – keep prefixes
nephritis – nephrectomy	Same root *nephr-* but inflammation versus removal	**Misleading** – do not conflate by root
medical – medication (Porter)	Profession versus drug	**Misleading**
postoperative – operate	Different time relation	Misleading – keep prefixes

Decision: conflate inflection always; conflate derivation only for suffixes that preserve the topic (-ion, -ation, -ive, -able, -ment, -ity, -ful, -ness); never strip prefixes (hyper-, hypo-, post-, pre-, re-, un-) and never conflate combining-form compounds by their shared root. This is the policy used by my 'full analysis' condition in Task 9.

TASK 7 – Finite-State Morphological Parsing	5 Marks

Ans:

A finite-state transducer relating surface forms to stems and features

1. The transducer as a two-tape automaton

A finite-state transducer (FST) is a finite automaton whose arcs carry a **pair** of symbols *a:b*. It reads one tape and writes the other, so it defines a **relation** between two strings instead of a yes/no language. In my system the **lexical tape** holds the stem and its features (for example fracture +N +PL) and the **surface tape** holds the spelling (fractures). The same arcs are read in either direction: reading the lexical tape and writing the surface tape is *generation*; reading the surface tape and writing the lexical tape is *parsing*.

Worked example. fracture +N +PL is read as the letters f, r, a, c, t, u, r, e and then the two feature symbols +N and +PL. The lexicon FST copies the letters and, at +PL, writes the intermediate string ^s# (a morpheme boundary, the plural s and a word end). The rule transducers (Task 8) then turn fracture^s# into fractures. Running the transducer backwards from fractures recovers fracture +N +PL.

2. The lexicon as an FST with continuation classes

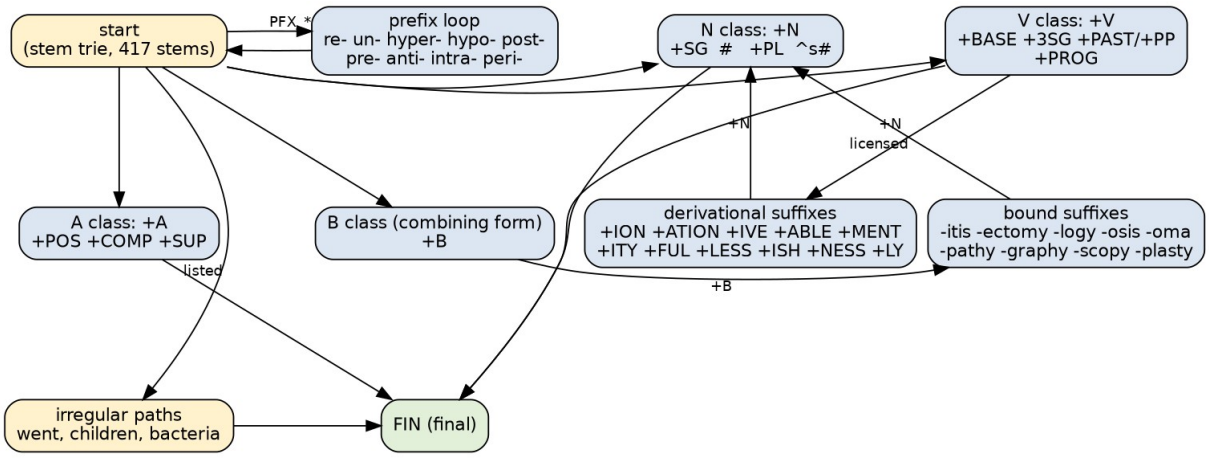


Figure 10: Lexicon FST organised into continuation classes

Table 34: Continuation classes of the lexicon FST

Class	Entered by	Arcs (lexical : intermediate)	Next
N (211 stems)	+N	+SG : # +PL : ^s#	FIN, or derivational suffix
V (150 stems)	+V	+BASE : # +3SG : ^s# +PAST, +PP : ^ed# +PROG : ^ing# (~ instead of ^ for stems that double)	FIN, or derivational suffix

Class	Entered by	Arcs (lexical : intermediate)	Next
A (78 stems)	+A	+POS : # +COMP : ^er# +SUP : ^est# (comparison only for listed adjectives)	FIN, or +NESS / +LY
B – combining forms (28)	+B	+ITIS : -itis, +ECTOMY : -ectomy, +LOGY : -logy, ... ; ε : - for B+B compounds (chole+cysto)	N class
Function words (101)	+FW	+FW : #	FIN
Irregular paths (92 forms)	stem letters	g:went, o:ε, +V:ε, +PAST:# (stem letters read, whole irregular surface written)	FIN
Prefix loop	start state	PFX_RE:re= PFX_UN:un= PFX_HYPER:hyper= ... (9 prefixes)	start state

The lexicon covers **three parts of speech plus bound forms** (nouns, verbs, adjectives, combining forms). It holds 419 stems, 23 inflectional affix variants and 32 derivational processes (Tasks 5 and 6). The explicit lexicon FST has 5697 states and 7055 arcs; composed with the seven rule transducers the single machine has **6736 states and 8863 arcs**.

3. Hand-traced parses of five surface forms

Each trace lists, for every step of the machine, the state before and after (named after the lexicon state), the symbols consumed on the **lexical tape** and the symbols emitted on the **surface tape**. These were read off the actual run of the composed transducer. Because the rule transducers hold a stem-final *e* until they see what follows, an *e* can be emitted a step late; this is visible in the traces.

Table 35: Parse of fractures → fracture +N +PL

State sequence (before → after)	Lexical tape (consumed)	Surface tape (emitted)
q0 → fracture	fracture	fractur
fracture → fracture+N	+N	ε
fracture+N → fracture+N.0	+PL	ε
fracture+N.0 → fracture+N.1	ε	e
fracture+N.1 → fracture+N.1	ε	s

Table 36: Parse of fractures → fracture +V +3SG

State sequence (before → after)	Lexical tape (consumed)	Surface tape (emitted)
q0 → fracture	fracture	fractur
fracture → fracture+V	+V	ε
fracture+V → fracture+V.0	+3SG	ε
fracture+V.0 → fracture+V.1	ε	e
fracture+V.1 → fracture+V.1	ε	s

Table 37: Parse of stopped → stop +V +PAST

State sequence (before → after)	Lexical tape (consumed)	Surface tape (emitted)
q0 → stop	stop	stop
stop → stop+V	+V	ε
stop+V → stop+V.0	+PAST	p
stop+V.0 → stop+V.1	ε	e
stop+V.1 → stop+V.2	ε	d

Table 38: Parse of stopped → stop +V +PP

State sequence (before → after)	Lexical tape (consumed)	Surface tape (emitted)
q0 → stop	stop	stop
stop → stop+V	+V	ε
stop+V → stop+V.0	+PP	p
stop+V.0 → stop+V.1	ε	e
stop+V.1 → stop+V.2	ε	d

Table 39: Parse of children → child +N +PL

State sequence (before → after)	Lexical tape (consumed)	Surface tape (emitted)
q0 → q0.0	c	c
q0.0 → q0.1	ε	h
q0.1 → q0.2	ε	i
q0.2 → q0.3	ε	l
q0.3 → q0.4	ε	d
q0.4 → q0.5	ε	r
q0.5 → q0.6	ε	e
q0.6 → irr:children/+PL:0	ε	n
irr:children/+PL:0 → irr:children/+PL:4	hild	ε
irr:children/+PL:4 → irr:child+N	+N	ε
irr:child+N → FIN	+PL	ε

Table 40: Parse of operations → operate +V +ION +N +PL

State sequence (before → after)	Lexical tape (consumed)	Surface tape (emitted)
q0 → operate	operate	operat
operate → operate+V	+V	ε
operate+V → operate+V.0	+ION	ε
operate+V.1 → operate+V.1	ε	i
operate+V.1 → operate+V.2	ε	o
operate+V.2 → DER_N	ε	n
DER_N → N_class	+N	ε
N_class → N_class.0	+PL	ε
N_class.0 → N_class.1	ε	s

Table 41: Parse of nephrectomy → nephro +B +ECTOMY +N +SG

State sequence (before → after)	Lexical tape (consumed)	Surface tape (emitted)
q0 → nephro	nephro	nephr
nephro → B_after	+B	ε
B_after → B_after.0	+ECTOMY	ε
B_after.1 → B_after.1	ε	e
B_after.1 → B_after.2	ε	c
B_after.2 → B_after.3	ε	t
B_after.3 → B_after.4	ε	o
B_after.4 → B_after.5	ε	m
B_after.5 → DER_N	ε	y
DER_N → N_class	+N	ε
N_class → FIN	+SG	ε

4. Generation with the same machine

The object that parsed the forms above is the same object that generates them. In code, M.up('fractures') returned ['fracture +N +PL', 'fracture +V +3SG'] and M.down(lexical('fracture +N +PL')) returned ['fractures'], with M the identical composed transducer of 6736 states; no second machine was built. Generation is the same arcs read left-to-right, parsing the same arcs read right-to-left.

Table 42: Generation from stem plus features

Lexical input	Surface output
fracture +N +PL	fractures
fracture +V +PROG	fracturing
stop +V +PAST	stopped
bleed +V +PAST	bled
operate +V +ION +N +PL	operations
PFX_POST operate +V +IVE +A +POS	postoperative
nephro +B +ECTOMY +N +PL	nephrectomies
chole +B cysto +B +ITIS +N +SG	cholecystitis

5. Ambiguity

Table 43: Surface forms with more than one analysis (all returned by the transducer)

Surface	All analyses
saw	saw +N +SG ; see +V +PAST
leaves	leaf +N +PL ; leave +V +3SG
drains	drain +N +PL ; drain +V +3SG
left	leave +V +PAST ; leave +V +PP
dressing	dress +V +PROG ; dressing +N +SG
fractures	fracture +N +PL ; fracture +V +3SG

Surface	All analyses
found	find +V +PAST ; find +V +PP
lead	lead +N +SG ; lead +V +BASE

The transducer returns **all** analyses of a form: *saw*, *leaves*, *drains* and *dressing* each have two or more, and *left* and *found* have two feature analyses (+PAST and +PP) of one lexeme. A downstream component chooses among them using context. A part-of-speech tagger (a probabilistic model, for example a hidden Markov model) would pick *see +V +PAST* for *saw* after a subject noun and *saw +N* after *the*; for search, the simplest policy is to index a record under every analysis, which raises recall at some cost to precision. The corpus has 72 ambiguous word types of 382 (19%).

TASK 8 – Orthographic Rules and Two-Level Morphology	4 Marks

Ans:

Spelling changes at morpheme boundaries

1. Why the lexicon transducer alone gives wrong spellings

Table 44: Plain concatenation of stem and suffix

Lexical form	Concatenation	Correct surface	Process missing
fracture + ing	fractureing	fracturing	e-deletion
stop + ed	stoped	stopped	consonant doubling
apply + s	applys	applies	y-to-ie
abscess + s	abscesss	abscesses	e-insertion
nephro + itis	nephroitis	nephritis	combining-vowel deletion
lie + ing	lieing	lying	ie-to-y

2. Orthographic rules in rewrite notation

Table 45: The seven rules (a → b / c __ d)

Rule	Notation	Context
R1 ie→y	ie → y / __ ^ i	before a suffix starting with i (lie+ing → lying)
R2 doubling	ε → (copy of last C) / V C __ ~	only after stems the lexicon marks with ~ (stop, admit, refer, big)
R3 e-deletion	e → ε / __ ^ V	stem-final e before a vowel-initial suffix
R4 y→i/ie	y → ie / C __ ^ s ; y → i / C __ ^ {e,n,f,l}	consonant + y before s (plural, 3sg) or e, n, f, l (ed, er, ness, ful, ly)
R5 e-insertion	ε → e / {x, s, z, sh, ch} ^ __ s #	sibilant-final stems before the s of plural or 3rd person
R6 o-deletion	o → ε / __ - V	combining vowel before a vowel-initial suffix (nephro-itis → nephritis)
R7 cleanup	{^, -, #, ~, =} → ε	remove all boundary symbols, applied last

3. Rule transducers and their composition

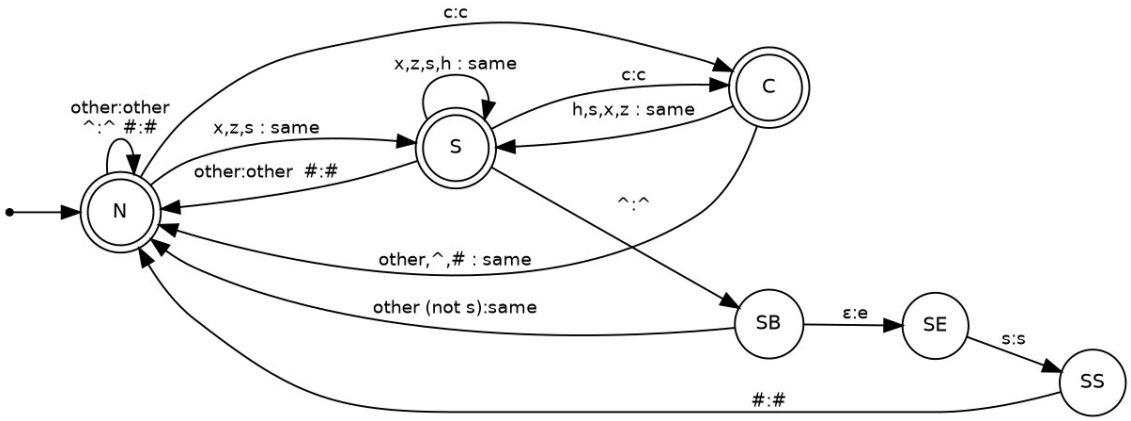


Figure 11: Transducer for rule R5 (e-insertion): arcs are input:output, other = any other symbol copied

Table 46: Size of each rule transducer (explicit states and arcs over the 33-symbol alphabet)

Rule transducer	States	Arcs
R1 ie->y	262	3491
R2 doubling	53	1045
R3 e-deletion	160	2329
R4 y->i/ie	142	1915
R5 e-insertion	6	130
R6 o-deletion	160	2329
R7 cleanup	1	32

Each rule is implemented as an explicit finite-state transducer (morph.boundary_rewrite, morph.from_step) and the cascade is obtained by **composition**: compose(L, R1), then with R2, and so on, giving one machine. R2 remembers the last consonant in its state, which is why it needs 53 explicit states after multi-symbol outputs are split; a textbook figure shows it as two states with a register.

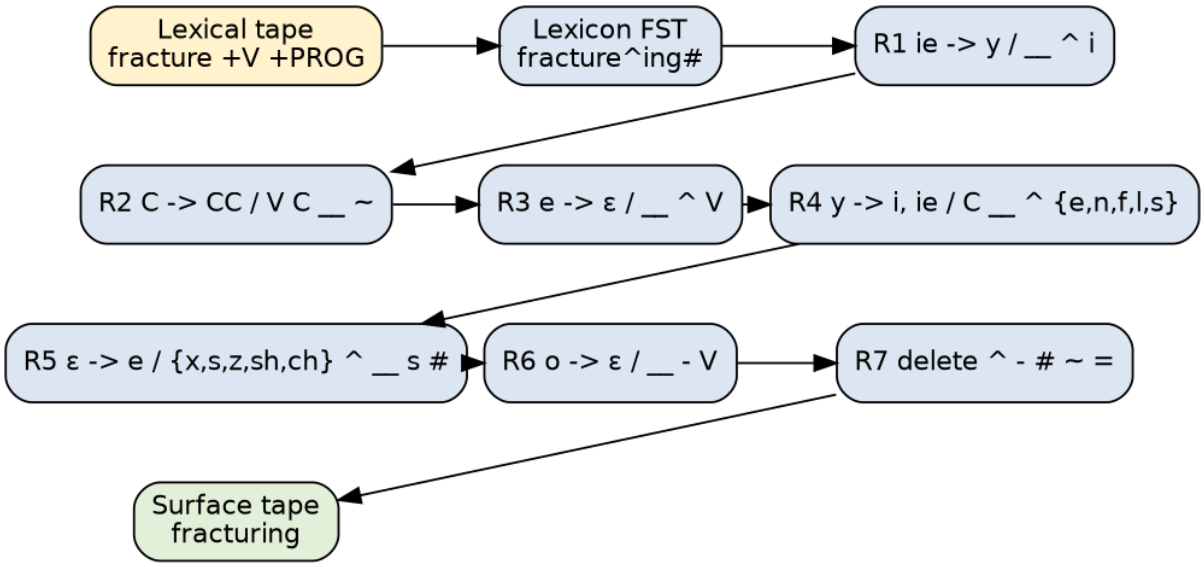


Figure 12: The cascade: lexicon transducer composed with seven rule transducers, top to bottom, left to right

4. Order dependencies

The order matters. I recomposed the machine with the order changed and compared the outputs:

Table 47: Output of the correct order and of a deliberately wrong order

Lexical form	Correct order	Wrong order	Order changed
die +V +PROG	dying	diing	R1 before R3 (correct) vs R3 before R1
lie +V +PROG	lying	liing	R1 before R3 (correct) vs R3 before R1
tie +V +PROG	tying	tiing	R1 before R3 (correct) vs R3 before R1
fracture +V +PROG	fracturing	fracturing	R1 before R3 (correct) vs R3 before R1
nephro +B +ITIS +N +SG	nephritis	nephroitis	R6 before cleanup vs cleanup before R6
chole +B cysto +B +ECTOMY +N +SG	cholecystectomy	cholecystoectomy	R6 before cleanup vs cleanup before R6
dress +V +3SG	dresses	dresss	R5 before cleanup vs cleanup first
abscess +N +PL	abscesses	abscesss	R5 before cleanup vs cleanup first
stitch +N +PL	stitches	stitchs	R5 before cleanup vs cleanup first

Constraints discovered: **R1 must precede R3** (otherwise *die+ing* loses its *e* first and R1 no longer applies); **R7 (cleanup) must be last** because R3, R5 and R6 need the boundary symbols; R4 and R5 commute because R4 produces an *ie* that is not a sibilant context; R2 does not interact with the others.

5. Cascade versus parallel two-level morphology

Cascade (what I built). Rules apply in sequence and each reads the full output of the previous one. Advantage: each rule is simple and independently testable, and rewrite notation is easy to write. Disadvantages: the order must be correct and intermediate strings exist that are neither lexical nor surface. **Two-level (Koskenniemi).** All rules are constraints on lexical:surface *pairs* and run simultaneously (their intersection is one machine); there is no intermediate string and no ordering to get wrong, and rules can be checked against both tapes at once. The price is that each constraint is harder to write, because rule contexts must be expressed over pairs, and rule conflicts show up as an intersection that rejects valid pairs. For a lexicon of this size the cascade is quicker to build; for a language with many interacting rules the two-level formulation scales better.

6. Testing the system on 30 forms

Table 48: Tests of the rule cascade (pure rules, then with the five listed exceptions)

Lexical form	Expected	Pure cascade		With exceptions	
stitch +N +PL	stitches	stitches	✓	stitches	✓
dress +V +3SG	dresses	dresses	✓	dresses	✓
abscess +N +PL	abscesses	abscesses	✓	abscesses	✓
mass +N +PL	masses	masses	✓	masses	✓
stomach +N +PL	stomachs	stomaches	✗	stomachs	✓
fracture +V +PROG	fracturing	fracturing	✓	fracturing	✓
infuse +V +PAST	infused	infused	✓	infused	✓
large +A +COMP	larger	larger	✓	larger	✓
see +V +PROG	seeing	seing	✗	seeing	✓
dye +V +PROG	dyeing	dying	✗	dyeing	✓
improve +V +MENT +N +SG	improvement	improvement	✓	improvement	✓
apply +V +3SG	applies	applies	✓	applies	✓
carry +V +PAST	carried	carried	✓	carried	✓
study +V +PROG	studying	studying	✓	studying	✓
dry +A +COMP	drier	drier	✓	drier	✓
shy +A +COMP	shyer	shier	✗	shier	✗
injury +N +PL	injuries	injuries	✓	injuries	✓
stop +V +PAST	stopped	stopped	✓	stopped	✓
admit +V +PROG	admitting	admitting	✓	admitting	✓
refer +V +PAST	referred	referred	✓	referred	✓
big +A +COMP	bigger	bigger	✓	bigger	✓
control +V +PAST	controlled	controlled	✓	controlled	✓
lie +V +PROG	lying	lying	✓	lying	✓
die +V +PROG	dying	dying	✓	dying	✓
nephro +B +ITIS +N +SG	nephritis	nephritis	✓	nephritis	✓
nephro +B +LOGY +N +SG	nephrology	nephrology	✓	nephrology	✓

Lexical form	Expected	Pure cascade		With exceptions	
colo +B +STOMY +N +SG	colostomy	colostomy	✓	colostomy	✓
chole +B cysto +B +ECTOMY +N +SG	cholecystectomy	cholecystectomy	✓	cholecystectomy	✓
be +V +PROG	being	bing	✗	being	✓
undergo +V +3SG	undergoes	undergos	✗	undergoes	✓

The pure cascade got **24 of 30** right; with the five lexicon exceptions it got **29 of 30**. Accounting for every failure of the pure cascade:

- **stomach +N +PL → stomachs** (pure output: stomaches): R5 over-generates: ch pronounced /k/, so no e is inserted. Fixed by listing *stomachs*.
- **see +V +PROG → seeing** (pure output: seing): R3 over-generates: stem-final ee. Fixed by listing *seeing*.
- **dye +V +PROG → dyeing** (pure output: dying): R3 over-generates: *dyeing* keeps the e to stay distinct from *dying*. Fixed by listing.
- **shy +A +COMP → shyer** (pure output: shier): R4 gives *shier*, which is acceptable but not the expected *shyer*; both are correct in English (accepted variation, not fixed).
- **be +V +PROG → being** (pure output: bing): R3 deletes the only vowel of *be*, giving *bing*. Fixed by listing *being*.
- **undergo +V +3SG → undergoes** (pure output: undergos): No rule inserts e after o: *undergos*. Fixed by listing *undergoes*.

TASK 9 – The Porter Stemmer	5 Marks

Ans:

A lexicon-free stemmer and the cost in precision

1. The algorithm

A letter is a **vowel** if it is a, e, i, o or u, or y preceded by a consonant; otherwise it is a **consonant**. Every word has the form [C](VC)^{m}[V], and **m** is the number of VC pairs (the *measure*). Conditions used in the rules: **condition v** – the stem contains a vowel; **condition d** – the stem ends in a double consonant; **condition o** – the stem ends consonant-vowel-consonant where the last consonant is not w, x or y. The algorithm has five steps: 1a plurals (sses→ss, ies→i, s→∅); 1b -eed, -ed, -ing with clean-up; 1c y→i; 2 and 3 map long suffixes to shorter ones when m>0; 4 removes suffixes when m>1; 5 removes a final e and a double l.

Table 49: Calculation of the measure m for eight clinical words

Word	C/V string	Collapsed	m
fracture	CCVCCVCV	CVCVCV	2
hypertension	CVCVCCVCCVVC	CVCVCVCVC	4
nephrectomies	CVCCCVCCVCVVC	CVCVCVCVC	4
dressing	CCVCCVCC	CVCVC	2
operative	VCVCVCVCV	VCVCVCVCV	4
medication	CVCVCVCVVC	CVCVCVCVC	4
inflammatory	VCCCVCCVCVCV	VCVCVCVCV	4
bleeding	CCVVCVCC	CVCVC	2

Table 50: Conditions v, d and o on test words

Word	v: has a vowel	d: ends in double consonant	o: ends consonant-vowel-consonant
fil	True	False	True
hopp	True	True	False
fail	True	False	False
tr	False	False	False
fall	True	True	False
hop	True	False	True
fracture	True	False	False
schedul	True	False	True

2. Hand-traced runs of all five steps

Table 51: Porter trace for fracture → fractur

Step	Rule fired	Before	After
5a	e->" (m>1 or m=1 and not *o)	fracture	fractur

Table 52: Porter trace for hypertension → hypertens

Step	Rule fired	Before	After
4	ion->" (m>1)	hypertension	hypertens

Table 53: Porter trace for nephrectomies → nephrectomi

Step	Rule fired	Before	After
1a	ies->i	nephrectomies	nephrectomi

Table 54: Porter trace for dressing → dress

Step	Rule fired	Before	After
1b	ing->" (*v*)	dressing	dress

Table 55: Porter trace for operative → oper

Step	Rule fired	Before	After
3	ative->" (m>0)	operative	oper

Table 56: Porter trace for medication → medic

Step	Rule fired	Before	After
2	ation->ate (m>0)	medication	medicate
3	icate->ic (m>0)	medicate	medic

Table 57: Porter trace for inflammatory → inflammatori

Step	Rule fired	Before	After
1c	y->i (*v*)	inflammatory	inflammatori

Table 58: Porter trace for bleeding → bleed

Step	Rule fired	Before	After
1b	ing->" (*v*)	bleeding	bleed

I wrote the stemmer from the published algorithm (porter.py) and compared it with NLTK's PorterStemmer(ORIGINAL_ALGORITHM) on 4,920 word forms

generated from the lexicon: they disagreed on one word of two letters (*is*), which my implementation leaves unchanged and NLTK reduces to *i*.

3. Stemming against lemmatisation on the annotated forms

I took the 195 annotated content-word forms that have a gold analysis (the annotated set of 215 forms also holds 20 deliberate out-of-lexicon words; 20 regular forms were generated from lexicon paradigms to reach the required size). For each form I computed the Porter stem of the form and the Porter stem of its **lemma** produced by the Task 7 transducer, and compared them.

Table 59: Agreement of Porter stemming with lemmatisation

Measure	Value
Forms compared	195
Stem of form = stem of lemma (agree)	180 (92.3%)
Diverge	15 (7.7%)

Table 60: Where stem and lemma diverge (under-stemming)

Form	Lemma	Porter stem of form	Porter stem of lemma
left	leave	left	leav
bled	bleed	bled	bleed
leaves	leaf	leav	leaf
kept	keep	kept	keep
gave	give	gave	give
diagnoses	diagnosis	diagnos	diagnosi
bronchi	bronchus	bronchi	bronchu
bacteria	bacterium	bacteria	bacterium
took	take	took	take
given	give	given	give
slept	sleep	slept	sleep
kept	keep	kept	keep
children	child	children	child
taken	take	taken	take

4. Over-stemming and under-stemming

Under-stemming fails to conflate related forms. On the annotated set the rate is **7.7%** (15 of 195); typical causes are irregular forms (*bled* vs *bleed*, *children* vs *child*, *bacteria* vs *bacterium*) and stem changes (*diagnosis, diagnose, diagnostic* → three stems; *inflammation, inflammatory, inflame* → three stems). Across the whole corpus vocabulary 22 of 136 word families split into more than one Porter stem.

Over-stemming conflates words that must stay distinct. Over the corpus vocabulary of 234 word types Porter produced 157 stems, and **6 stems merge words of different families**. Examples:

Table 61: Stems that merge different families

Porter stem	Families merged
nephrectomi	nephro+B+ECTOMY+N, nephro+B+ECTOMY+N+PL, nephro+B+ECTOMY+N+SG
organ	organ, organism
hepat	hepatic, hepato+B+ITIS+N, hepato+B+ITIS+N+SG
nephriti	nephro+B+ITIS+N, nephro+B+ITIS+N+SG
medic	medicate, medicine
leav	leaf, leave

On the annotated set the over-stemming rate is **6.7%**. The clinically dangerous case is *medical* and *medication* sharing the stem *medic*, so a search for *medical* returns drug records.

5. The retrieval experiment

I ran the same 40 queries three times: with no stemming (exact lower-case match after normalisation), with the Porter stemmer applied to query and records, and with full morphological analysis (the transducer's analyses mapped to lexicon families, with the conflation policy of Task 6). A fourth row shows inflection-only lemmatisation. Relevance judgments are by concept families I assigned, including cross-vocabulary links such as *kidney* with *renal*, *nephritis*, *nephrectomy*.

Table 62: Precision, recall and F1 (macro-average over 40 queries)

Condition	Precision	Recall	F1
No stemming	0.939	0.575	0.656
Porter stemmer	0.890	0.817	0.800
FST lemma only (inflection)	0.934	0.782	0.806
Full morphological analysis	0.939	0.949	0.921

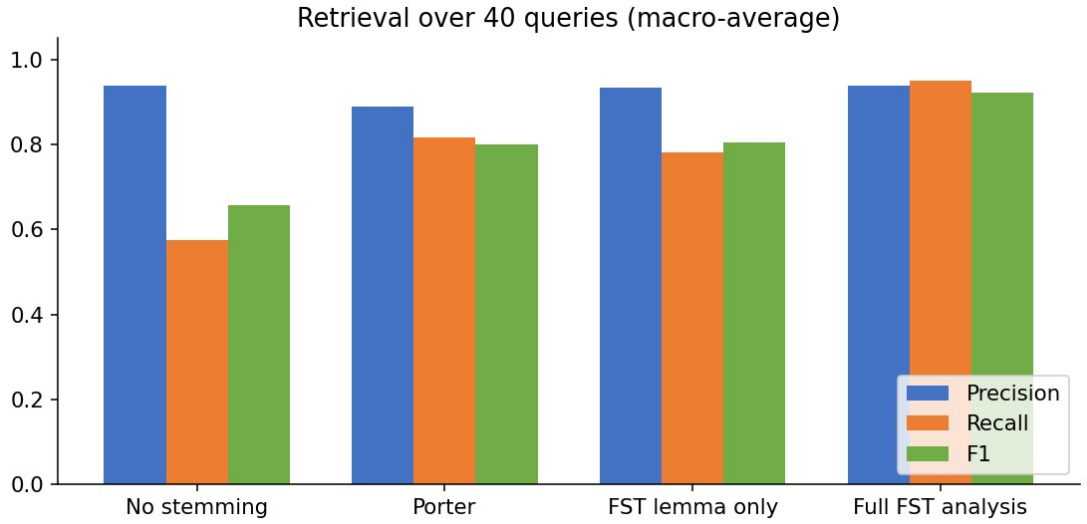


Figure 13: Retrieval quality of the four conditions

Table 63: F1 by query group

Group (queries)	No stemming	Porter	FST lemma	Full analysis
inflection	0.768	0.983	0.983	0.983
derivational family	0.591	0.702	0.714	0.948
cross-vocabulary concept	0.603	0.727	0.727	0.832
over-stemming trap	0.596	0.672	0.688	0.866

Porter gives a large recall gain over exact matching, but precision falls on the over-stemming queries (*medical*, *organ*). Full analysis scores highest because it handles irregular forms and keeps unrelated families apart. Its remaining errors are ambiguous forms (an analysis for each reading is indexed) and queries whose relevance needs a medical link the lexicon does not contain (*hepatic* with *liver* when only a combining-form root links them).

TASK 10 – Implement and Test the Pipeline	3 Marks

Ans:

Implementation, results and test procedure

1. Implementation

Table 64: Modules (Python 3.12, standard library plus NLTK for cross-checking only)

Module	Contents
tokenizer.py	maximal-munch tokenizer, abbreviation/date/unit normaliser
patterns.py, evalpat.py	12 entity regular expressions; exact-span evaluation
automata.py	DFA, NFA, subset construction, Moore minimisation, Thompson construction, state elimination, validators
lexicon.py	417 stems, inflectional and derivational affixes, irregular list
morph.py	explicit-arc FST, composition, lexicon FST, 7 rule transducers, parse (up) and generate (down)
porter.py	Porter stemmer with rule trace
corpus.py, exp1.py, extra.py, exp12.py	corpus generator, experiments for Tasks 2–11 and Task 12

Three excerpts show the core algorithms. The subset construction:

```
def subset(s):
    start=s.eclose({s.start}); ids={start:"D0"}; order=[start]; rows=[]; delta={}; q=[start]
    while q:
        S=q.pop(0); row=[ids[S],"{"+",".join(sorted(S,key=lambda x:(len(str(x)),str(x))))+"}"]
        for a in s.alpha:
            T=s.eclose(s.move(S,a))
            if not T: row.append("∅"); continue
            if T not in ids: ids[T]="D%d"%len(ids); order.append(T); q.append(T)
            row.append(ids[T]); delta[(ids[S],a)]=ids[T]
        rows.append(row)
    fin=[ids[S] for S in ids if S&s.finals]
    return DFA(s.name+" subset",s.alpha,"D0",fin,delta,s.classify),rows
```

The measure m of the Porter stemmer:

```
def measure(stem):
    f=form(stem);
    # collapse runs, count VC pairs
    r=[];
    for ch in f:
        if not r or r[-1]!=ch: r.append(ch)
    s="".join(r)
    if s.startswith("C"): s=s[1:]
    return s.count("VC")
```

Composition of two transducers (the step that builds one machine from the lexicon and the rules):

```
def compose(A,B,name="composed"):
```

```
    """A: x->y, B: y->z  gives x->z.  Both normalised (outputs of length <=1)."""
    C=FST(name); smap={(A.start,B.start):0}; C.label[0]=(A.label[A.start],B.label[B.start])
    Bi=B._byin(); q=collections.deque([(A.start,B.start)])
    def get(k):
        if k not in smap:
            c=C.new((A.label[k[0]],B.label[k[1]])); smap[k]=c; q.append(k)
            if k[0] in A.finals and k[1] in B.finals: C.finals.add(c)
        return smap[k]
    if A.start in A.finals and B.start in B.finals: C.finals.add(0)
    while q:
        a,b=q.popleft(); c=smap[(a,b)]
        for (i,o,da) in A.arcs[a]:
            if o=="":
                C.add(c,i,"",get((da,b)))
            else:
                for (o2,db) in Bi[b].get(o,[]): C.add(c,i,o2,get((da,db)))
        for (o2,db) in Bi[b].get("",[]): C.add(c,"",o2,get((a,db)))
    C._bi=None; C._bo=None
    return C.trim()
```

2. Whole-pipeline run over the corpus

Table 65: Items processed, matched or analysed, rejected, and run-time per stage

Stage	Processed	Matched / analysed	Rejected	Time (s)
Tokenizer	520 records	46748 tokens	–	0.164
Entity patterns (12)	520 records	5369 matches	–	0.079
Automata validators	1393 matched strings	1381 accepted	12 rejected	<0.05
Normaliser (+ tokenise words)	520 records	644 abbreviations, 188 dates, 153 units	–	0.232
Morphological analyser (FST)	382 word types (17135 tokens)	316 types analysed; token coverage 98.5%	66 types	0.020
Porter stemmer	382 word types	all stemmed	0	0.005

Of the 382 word types, 66 have no analysis; their commonest members are the out-of-lexicon symptom words I inserted deliberately (dysuria, epistaxis, icterus, oedema, pallor, erythema).

Table 66: Automata run on the strings the patterns extracted

Validator	Accepted	Rejected	Rejected strings are
PID DFA	499	0	none: all matches have the format
ICD DFA	173	12	codes with a trailing letter (S72.001A) – outside the core validator
BP DFA	365	0	none
Date DFA	108	0	non-slash numeric dates after conversion
Phone NFA	236	0	none (lab accession numbers pass because they have the same shape)

3. Test cases and comparison with hand results

Table 67: Automaton tests (hand-computed expectation from Task 4 versus program output)

Automaton	String	Expected	Program	Match
PID DFA	PID-2024-045321	accept	accept	✓
PID DFA	PID-2024-04532	reject	reject	✓
ICD DFA	N18.3	accept	accept	✓
ICD DFA	N1	reject	reject	✓
BP DFA	120/80	accept	accept	✓
BP DFA	1200/80	reject	reject	✓
Date DFA	31/12/2023	accept	accept	✓
Date DFA	32/01/2024	reject	reject	✓
Date DFA	12/13/2024	reject	reject	✓
Phone NFA	+91 98765 43210	accept	accept	✓
Phone NFA	98765 4321	reject	reject	✓

Table 68: One string that must be rejected for each pattern

Pattern	Test string	Whole-string match?	Result
PID	PID-2024-04532	no	rejected ✓
DATE	Admitted on 32/01/2024	no	rejected ✓
DRUG	Hb 11.2 g/dL	no	rejected ✓
FREQ	ODD	no	rejected ✓
LAB	Hb 11.2	no	rejected ✓
BP	12/03/2024	no	rejected ✓
TEMP	Na 138 mmol/L	no	rejected ✓
ICD	1B2	no	rejected ✓
WARDBED	Bed 12	no	rejected ✓
PHONE	987654321	no	rejected ✓
AGE	for 5 years	no	rejected ✓
DURATION	after 2 weeks	no	rejected ✓

Table 69: Irregular forms (Task 5) and ambiguous forms (Task 7)

Surface	Expected analysis (hand)	Program output	Match
went	go +V +PAST	go +V +PAST	✓
children	child +N +PL	child +N +PL	✓
bacteria	bacterium +N +PL	bacterium +N +PL	✓
bled	bleed +V +PAST	bleed +V +PAST; bleed +V +PP	✓
diagnoses	diagnosis +N +PL	diagnose +V +3SG; diagnosis +N +PL	✓
taken	take +V +PP	take +V +PP	✓
saw	2 or more analyses (Task 7)	saw +N +SG; see +V +PAST	✓
leaves	2 or more analyses (Task 7)	leaf +N +PL; leave +V +3SG	✓
drains	2 or more analyses (Task 7)	drain +N +PL; drain +V +3SG	✓

Table 70: Porter results (hand traces of Task 9 versus program and NLTK)

Word	Hand trace	Program	NLTK original	Match
fracture	fractur	fractur	fractur	✓
hypertension	hypertens	hypertens	hypertens	✓
nephrectomies	nephrectomi	nephrectomi	nephrectomi	✓
dressing	dress	dress	dress	✓
operative	oper	oper	oper	✓
medication	medic	medic	medic	✓
inflammatory	inflammatori	inflammatori	inflammatori	✓
bleeding	bleed	bleed	bleed	✓

4. Result plots

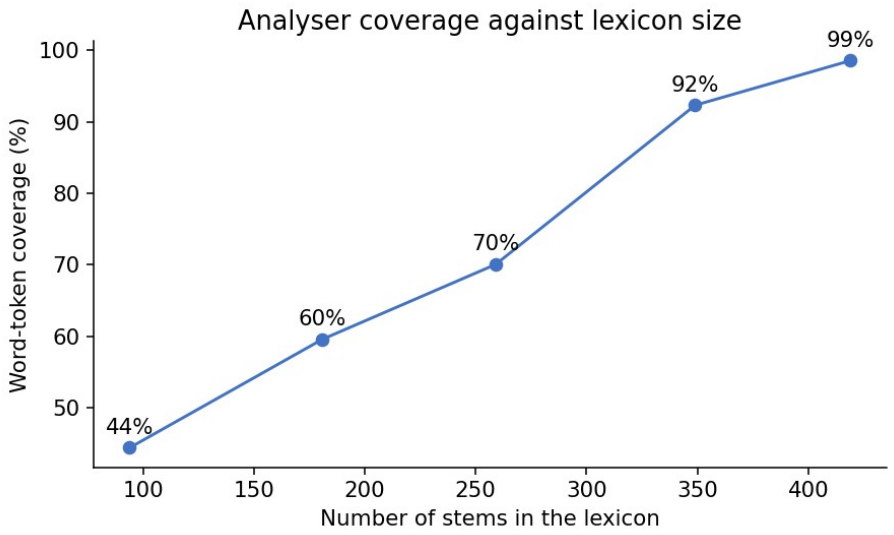


Figure 14: Analysis coverage against lexicon size (random subsets of the 465 lexicon entries)

Coverage rises from 44.4% with 94 stems to 98.5% with 419 stems, with the curve flattening because the remaining uncovered tokens are the deliberate out-of-lexicon words. Real text would keep rising far longer (Zipf's law).

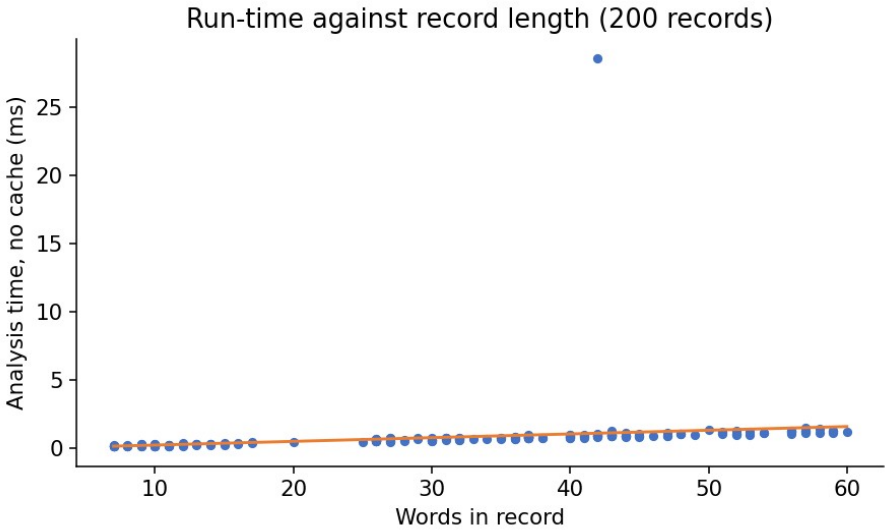


Figure 15: Run-time of analysis against record length (200 records, no caching)

The time grows linearly with the number of words (about 0.02 ms per word, fit slope), as expected for depth-first search over a deterministic-looking transducer whose branching is limited by the lexicon.

TASK 11 – Compare, Recommend and Defend the Normalisation Design	2 Marks

Ans:

Which normalisation design to deploy

1. Comparison from my own measurements

Table 71: No normalisation, Porter stemming and full finite-state analysis

Criterion	No normalisation	Porter stemmer	Full FST analysis
Retrieval F1 (P / R)	0.656 (0.939 / 0.575)	0.800 (0.890 / 0.817)	0.921 (0.939 / 0.949)
Coverage of word tokens	100% (nothing is analysed)	100% (rules apply to any string)	98.5% (out-of-lexicon words are left unanalysed)
Run-time, 520 records	tokenising only: 0.16 s	+ 0.005 s for the 382 word types	+ 0.020 s for the 382 word types (cached)
Maintenance effort	none	none (fixed rules)	high: lexicon, irregular list, rule cascade
Typical failure	*fractures* does not match *fracture*	*medical* matches *medication*; *bled* does not match *bleed*	unknown word is not analysed; ambiguous form indexes two readings

2. Recommendation

I recommend **full finite-state analysis with a Porter fallback for words the lexicon does not know**: analyse each word with the transducer; if it has no analysis, index its Porter stem. Highest recall alone is not the criterion (Porter's recall of 0.817 is close to the transducer's 0.949, but its precision is lower); what matters is that a clinician is not shown unrelated records.

Table 72: Expected accuracy, failure modes and what a clinician would notice

Failure mode	Cause	What the clinician notices
Missing records for unusual words	Word not in lexicon and its Porter stem differs from the query's	A relevant note does not appear; the clinician does not know it is missing
Extra records for ambiguous forms	*leaves*, *saw*, *drains* indexed under every reading	A few irrelevant hits at the bottom of the list
Relational words not linked	*hepatic* with *liver* needs a lexicon link	A search for *liver* misses notes written *hepatic*
Negation	*no fever* contains *fever*	Notes that say the patient has no fever are returned

Expected accuracy on clean text: F1 about **0.921** (precision 0.939, recall 0.949) on my 40 queries. The corpus and queries are synthetic, so on real records I would expect lower coverage and I would re-measure before relying on this figure.

3. Manual effort and the economic point of a learned approach

Table 73: Effort estimate (person-hours; planning estimates from my working rates, to be replaced by logged time)

Activity	Quantity	Rate	Hours
Writing and testing 12 patterns	12 patterns	30 min each	6
Designing automata and diagrams	5 machines	1 h each	5
Lexicon construction	417 stems	1.5 min per stem	10.4
Affixes, irregular forms, rules	~150 items	3 min each	7.5
Annotating word forms	215 forms	1 min each	3.6
Query set and relevance judgments	40 queries	8 min each	5.3
Corpus design and gold labelling	520 records	script plus checks	6
Total			**43.8**

A learned lemmatiser or tagger needs thousands of annotated forms; at 1 minute per form, 5,000 forms cost about 83 hours, a fixed cost. My lexicon costs about 1.5 minutes per new stem. The two lines cross at about 83 h ÷ 1.5 min ≈ **3,300 stems**, so **a statistical or learned approach becomes more economical once the vocabulary to cover passes a few thousand stems, or once more than about a fifth of the tokens stay out of lexicon** (the situation of Task 12), because hand-built coverage grows linearly with effort while a learned model generalises to unseen forms.

DAY-4 DESIGN CHALLENGE – TASK 12: Re-evaluate and Adapt the Design	8 Marks

Ans:

OCR output and Romanised regional-language notes

A. Impact analysis (2 marks)

The two new sources are described by three kinds of change: **character-level noise** (OCR confuses 0/O, 1/l/I, rn/m, cl/d), **layout noise** (words split across line ends with a hyphen) and **a second language** (Romanised Tamil written in the same notes, for example *patient ku fever irundhuchu, paracetamol 500mg kuduthen*). I built a test sample of 60 noisy notes: 20 OCR notes (a 4% character-error model applied to 20 corpus records), 20 code-mixed notes, and 20 code-mixed notes that were then OCR-corrupted.

Table 74: What the original components cannot handle

Component	New-input property it fails on	Example
Entity patterns	digit/letter confusion inside numbers; a line-break hyphen inside a unit or word	*5OO mg*, *12O/8O mmHg*, *PID-2O24-O45321*
Automata validators	they accept or reject exactly, with nothing in between	a single *O* for *0* rejects the whole identifier
Lexicon and analyser	out-of-lexicon stems; non-English morphology; noise creates non-words	*u1cer*, *Speci-\nmen*, *irundhuchu*
Tokenizer / normaliser	token-level code-mixing: no language label per token	*ku*, *kuduthen* are treated as English words
Stemmer	assumes English suffixes	*irundhuchu* is stemmed as if it were English

Table 75: Assumptions that no longer hold

Assumption	Why it breaks
Clean typed input	OCR substitutes characters inside words and numbers
Closed vocabulary	at least 20% of tokens are out-of-lexicon, misspelt or non-English (measured below)
English-only morphology	Tamil suffixes (*-uchu*, *-en*) are not in the lexicon
Exact matching	a single wrong character rejects a valid entity

Estimate from the clean-corpus error rates. If an entity survives only when none of its confusable characters is corrupted, its expected recall falls from the clean recall r to r·(1−p·k)^c, where c is the average number of confusable characters per entity (counts in the corpus: 4.5 for blood pressure, 3.7 for identifiers, 4.4 for phone numbers) and p·k is the corruption probability per character (4% base rate, scaled for digits). With the Task 2 recalls this predicts

an overall pattern recall of **0.769** at a 4% error rate, down from 0.923. For words the analyser's coverage should fall from 98.5% to about **94.0%** (the clean coverage multiplied by the probability that a word has no corrupted letter). I measured the effect below to test the prediction.

B. Evaluation of the original design (2 marks)

Three records through the original pipeline. For each I recorded what fails first.

OCR note. OPERATION NOTE. PID-2024-125544. Procedure: mastectomy. Indication: ulcer . Anaesthesia: local. The joint was incised and incision was closed with sut… First failure: the entity patterns miss *BP I54/93 mmHg*; the analyser then leaves these words unanalysed: *placecl*, *speci*, *moni*, *toring*, *musc*.

code-mixed note. pt ku fever bleeding irundhuchu, Inj Furosemide 30mg 0-0-1 kuduthen. dressing nalla irukku, sapadu saapidanum. for 8 months swelling irundhuchu, romba… First failure: the entity patterns miss none; the analyser then leaves these words unanalysed: *ku*, *irundhuchu*, *kuduthen*, *nalla*, *irukku*, *sapadu*.

both note. pt ku cough cough irundhuchu, Inj Ranitidine 600 mg 1-0-1 kuduthen. for two days cough irundhuchu, romba konjam pain kammi. Notes: vertebra, cardiac.… First failure: the entity patterns miss *for two days*; the analyser then leaves these words unanalysed: *ku*, *irundhuchu*, *kuduthen*, *irundhuchu*, *romba*, *konjam*.

In the OCR note the first thing that fails is the pattern layer (a digit replaced by a letter breaks the numeric entity); in the code-mixed note the patterns are fine, but the analyser cannot analyse the Tamil words; in the combined note both fail, and a line-break hyphen splits an English word so that neither of its halves is in the lexicon.

Table 76: Degradation of the original pipeline (retrieval F1 = none / Porter / full analysis)

Input	Pattern P	Pattern R	Analyser coverage	Retrieval F1
Clean corpus (baseline)	0.955	0.924	98.5%	0.656 / 0.800 / 0.921
OCR (20 notes)	0.882	0.728	84.7%	0.670 / 0.828 / 0.863
code-mixed (20 notes)	1.000	0.966	46.0%	0.704 / 0.926 / 1.000
both (20 notes)	0.970	0.756	39.9%	0.585 / 0.675 / 0.758
All 60 noisy notes	0.932	0.789	63.0%	0.597 / 0.759 / 0.894

To separate the effect of noise from the effect of having only 60 documents, I also scored the same 60 records **without** corruption: retrieval F1 was 0.658 / 0.809 / 0.965 (none / Porter / full analysis) and coverage 71.4% (the Tamil

words are already out of lexicon). The measured OOV rate of the noisy sample is **37.0%** of word tokens, above the 20% the scenario expects.

Pattern recall fell from 0.924 to 0.728 on OCR notes (the prediction was 0.769) and coverage fell from 98.5% to 63.0% on the combined sample. **Different formalisms would be cheaper than extending what I have:** a *weighted edit-distance* match against a dictionary of word forms handles character substitutions with one small table of costs (rn↔m costs 0.3, not 2), instead of enlarging the lexicon with every misspelling; a *character n-gram language identifier* labels each token as English or regional with no lexicon of regional words at all; a *weighted automaton* would let the validators accept 'almost valid' identifiers with a cost instead of rejecting them.

C. Adaptation or redesign (2 marks)

Decision: MODIFY (keep the finite-state core; add a repair stage in front and a probabilistic stage beside it). A full redesign is not justified because the same lexicon, rule cascade and patterns still work on every clean token; the new problems are at the input boundary.

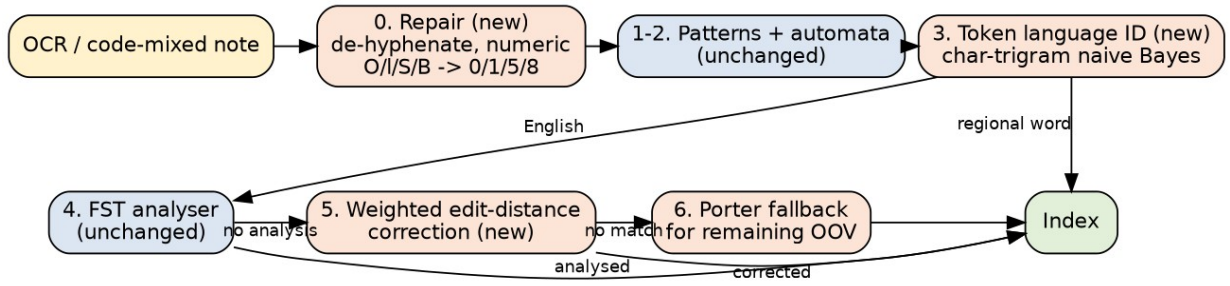


Figure 16: Revised pipeline: new stages shaded

Table 77: Exactly what changes

Part	Change
Patterns	unchanged; they now run on repaired text. Numeric repair maps O→0, l/I→1, S→5, B→8 only inside tokens that are mostly digits
Automata	unchanged (exact validators remain exact); the repair stage supplies near-valid strings
Lexicon	unchanged; a full-form dictionary of 1523 surface forms is generated from the transducer and used as the correction target; no regional stems are added to the English lexicon
Rule cascade	unchanged
New: de-hyphenation	(letter)-\n(letter) is joined
New: token language ID	a character-trigram naive-Bayes classifier trained on 40 romanised regional words and the English form dictionary; a token the analyser cannot analyse and the classifier labels regional is kept as is
New: weighted edit-distance correction	OCR confusion pairs cost 0.3–0.5, other edits 1; a word is replaced by the nearest dictionary form within cost 0.9 (0.6 for short words)
New: Porter fallback	remaining out-of-lexicon English tokens are indexed by their Porter stem

Table 78: Effect of the revised design (original → revised)

Input	Pattern P	Pattern R	Coverage (analysed → handled)	Retrieval F1
OCR	0.882 → 0.951	0.728 → 0.854	84.7% → 98.1%	0.863 → 0.879
code-mixed	1.000 → 1.000	0.966 → 0.966	46.0% → 94.5%	1.000 → 1.000
both	0.970 → 0.973	0.756 → 0.826	39.9% → 94.2%	0.758 → 0.980
all noisy	0.932 → 0.968	0.789 → 0.874	63.0% → 96.1%	0.894 → 0.949

Language ID: 82.1% accuracy over 871 tokens of the code-mixed notes; recall on Tamil words seen in training 100.0%, on unseen Tamil words 0.0%; 156 of 476 English tokens were wrongly labelled regional. **Correction:** on 212 words corrupted at a very heavy 50% rate the corrector restored the intended word 55.2% of the time.

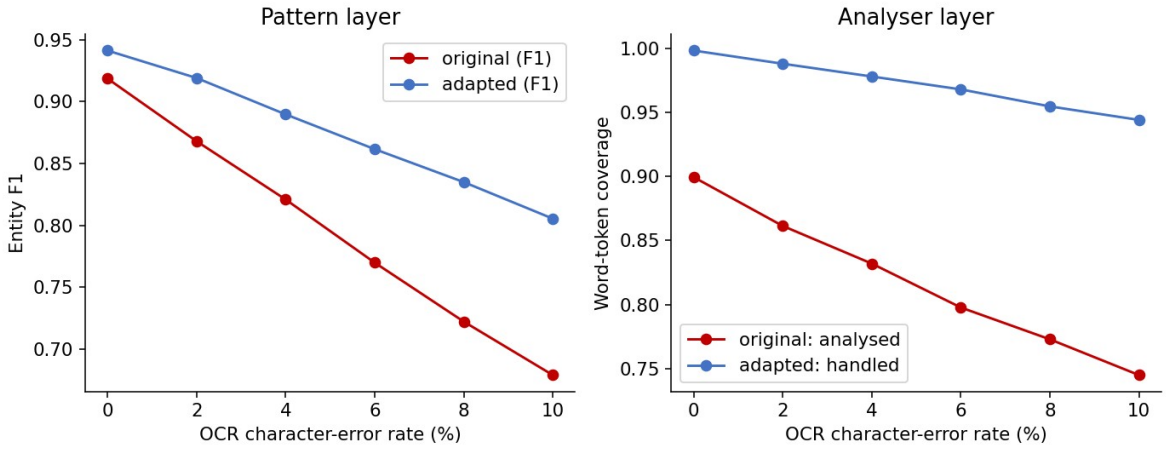


Figure 17: Entity F1 and word coverage against OCR character-error rate, original versus adapted

Table 79: Run-time and annotation effort of the revised design

Item	Effect
Run-time	correction searches the form dictionary only for words that fail analysis, with an edit-distance cache; 0.2 s for the 60 notes, against 0.02 s for the clean corpus analysis
Precision	pattern precision is unchanged; coverage rises because noisy and regional tokens are now routed
Annotation effort	about 54 regional words (45 min), confusion weights (30 min) and 60 test notes with gold labels (3 h): roughly 4 hours, against 44 hours for the original system

D. Evidence-based defence (2 marks)

Why MODIFY follows from the evidence. (1) Task 2 showed that every pattern error was a format variant or a cross-type homograph, not a defect of the formalism, so the patterns stay and only their input changes. (2) Task 7 and 8 showed that the transducer is correct on every analysed word (195 of 195 gold analyses found), so a wrong analysis is not the problem; unanalysed words are.

(3) The measurements above show the damage is concentrated at the boundary: OCR pattern recall 0.924 → 0.728 and coverage 98.5% → 63.0% are recovered to 0.854 and 96.1% by repair, correction and language identification. (4) Retrieval needs a decision: the Task 9 experiment shows Porter alone loses precision, so the fallback is used only for words the transducer cannot analyse.

Remaining limitations, stated honestly. The adapted design recovers pattern recall only partly, because corruption that changes the *shape* of an entity (a lost digit) cannot be repaired by substituting characters. The language identifier is weak on unseen regional words (recall 0.0%) because it is trained on only 40 words; a larger list or a classifier trained on real code-mixed notes is needed. Correction is unreliable when a word is heavily corrupted (55.2%). The evaluation sample is small (60 notes) and the code-mixed notes are synthetic, so the figures show direction and size, not exact values.

What a finite-state layer will never handle, however far it is extended: unbounded nesting and long-distance agreement (Task 4's pumping-lemma proof); the *meaning* of a note, such as negation (*no fever*), hedging and the scope of a diagnosis; context-dependent ambiguity such as *MS* or *leaves*, which needs a probabilistic model that looks at neighbouring words; and an open vocabulary of regional words and misspellings, which a hand-built lexicon can approach only at linear cost. The finite-state layer should therefore stay the fast, auditable normalisation stage, with probabilistic models placed beside it for the decisions it cannot make.